Lec04：面向对象经典的设计原则 - SOLID

Classic design principles of object-oriented programming - SOLID

课程编号：22WHSE22006（2023版培养计划）

授课对象：23 级软件工程、服务科学与工程专业

主讲教师：辛国栋

Email：xinguodong@163.com



QQ 群：软件构造2025

群　号：311528424

本章内容

	设计原则	一句话归纳	目的
S	单一职责原则 Single Responsibility Principle， SRP	类的职责要单一，不能将太多的职责放在一个类中	提高类的内聚性，实现代码的高内聚、松耦合
O	开闭原则 Open Closed Principle， OCP	软件实体对扩展是开放的，但对修改是关闭的，即在不修改一个软件实体的基础上去扩展其功能	降低新风险
L	里氏代换原则 Liskov Substitution Principle， LSP	子类对象能够替换程序中父类对象出现的任何地方，并且保证原来程序的逻辑行为不变及正确性不被破坏。	防止继承泛滥
I	接口隔离原则 Interface Segregation Principle， ISP	各个类建立自己的专用接口，而不是建立万能接口	功能解耦
D	依赖倒置原则 Dependence Inversion Principle， DIP	面向接口编程，而不是面向实现编程	有利于结构的升级扩展

1. 单一职责原则

Single Responsibility Principle，SRP

1.1 单一职责原则的含义

- 单一职责原则SRP的概念：A class or module should have a single responsibility 一个类或者模块只负责完成一个职责（或者功能）。
  - 一个类包含了两个或者两个以上业务不相干的功能，那么该类的职责不够单一，应该将它拆分成多个功能更加单一、粒度更细的类。
  - 一个类（或者大到模块，小到方法）承担的职责越多，它被复用的可能性越小，而且如果一个类承担的职责过多，就相当于将这些职责耦合在一起，当其中一个职责变化时，可能会影响其他职责的运作。
- 单一职责原则是实现高内聚、低耦合的 指导方针。
- 单一职责原则是最简单但又最难运用的原则，需要设计人员具有较强的分析设计能力和相关重构经验。在真实的软件开发中，类是否职责单一， 需要根据具体的应用场景、不同的业务层面判断

1.2 如何判断类的职责是否足够单一

```
public class UserInfo {

    private long userId;
    private String username;

    private String email;

    private String telephone;

    private long createTime;
    private long lastLoginTime;
    private String avatarUrl;

    //用户的地址信息
    private String provinceOfAddress;
    private String cityOfAddress;
    private String regionOfAddress;
    private String detailedAddress;
    //其他代码省略
}
```

应用场景1：单纯的社交产品中，用户的地址信息只有一份，此时 UserInfo 就是合理的

应用场景2：电商网站，用户的地址还要用在物流中，那最好将地址信息从 UserInfo 中拆分出来，独立成用户收货地址

应用场景3：账号在同一公司中多个产品中登录。需要继续对 UserInfo 拆分，将身份认证相关的信息（email、telephone 等）抽取成独立的类

结论：不同的应用场景、不同阶段的需求背景下，对同一个类的职责是否单一的判定，可能不一样。

1.3 类的职责是否设计得越单一越好?

```java
public class Serialization {
    private static final String IDENTIFIER_STRING = "UEUEUE;";
    private Gson gson;

    public Serialization() {
        this.gson = new Gson();
    }
    public String serialize(Map<String, String> object) {
        StringBuilder textBuilder = new StringBuilder();
        textBuilder.append(IDENTIFIER_STRING);
        textBuilder.append(gson.toJson(object));
        return textBuilder.toString();
    }
    public Map<String, String> deserialize(String text) {
        if (!text.startsWith(IDENTIFIER_STRING)) {
            return Collections.emptyMap();
        }
        String gsonStr = text.substring(IDENTIFIER_STRING.length());
        return gson.fromJson(gsonStr, Map.class);
    }
}
```

Serialization 类实现了一个简单协议的序列化和反序列功能。

根据单一职责原则，拆成两个类，一个是序列化，另一个反序列化，是否更好?

1.3 类的职责是否设计得越单一越好?

```
public class Serializer {
    private static final String IDENTIFIER_STRING = “UEUEUE;”;
    private Gson gson;
    public Serializer() {
        this.gson = new Gson();
    }
    public String serialize(Map<String, String> object) {
        // 代码略…
    }
}
```

需修改

拆分之后，Serializer 类和 Deserializer 类的职责更加单一了，但也随之带来了新的问题

例如：数据标识从“UEUEUE”改为“DFDFDF”

需修改

```
public class Deserializer {
    private static final String IDENTIFIER_STRING = "UEUEUE;";
    private Gson gson;
    public Deserializer() {
        this.gson = new Gson();
    }
    public Map<String, String> deserialize(String text) {
        // 代码略…
    }
}
```

如果只改了一个类的数据标识忘了更改另一个类，会导致序列化、反序列化不匹配，程序运行出错：

拆分之后，代码的可维护性变差；代码的内聚性显然没有拆分之前高。

1.4 基于单一职责原则重构练习 - 当前类代码

```
public class Order {
    public void createOrder(OrderInfo info) {
        // 订单创建逻辑
    }
    public void processPayment(PaymentInfo paymentInfo) {
        // 支付处理逻辑
    }
    public void trackLogistics(String orderId) {
        // 物流跟踪逻辑
    }
}
```

判断电商订单系统中类Order的设计是否符合单一职责原则 SRP?

此设计下，Order类承担多个职责。

- 支付方式新增（如支持数字货币支付），需修改processPayment 方法；
- 物流跟踪接口变更，需修改 trackLogistics 方法。

这些修改可能影响订单创建功能，因它们在同一类中

结论：

职责不单一，违反单一职责原则。修改其中一个职责可能影响其他职责，增加系统不稳定性和维护成本。

1.4 基于单一职责原则重构练习 - 代码浅度重构

```java
public class Order {
    public void createOrder(OrderInfo info) {
        // 订单创建逻辑
    }
}
```

拆分1

```java
public class Payment {
    public void processPayment(PaymentInfo paymentInfo) {
        // 支付处理逻辑
    }
}
```

拆分2

```java
public class Logistics {
    public void trackLogistics(String orderId) {
        // 物流跟踪逻辑
    }
}
```

拆分3

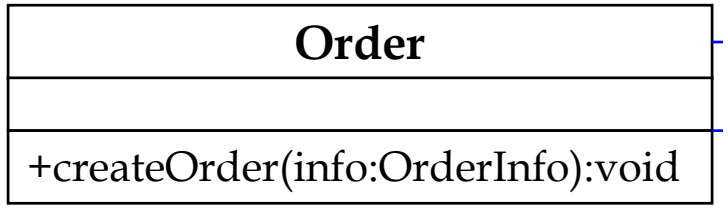
这种职责分离的拆分还是浮于表面，属于代码浅度重构　如何进行深度重构？这就涉及到设计问题

1.4 基于单一职责原则重构练习 – 代码深度重构

支付方式、物流跟踪代码变更影响订单创建功能

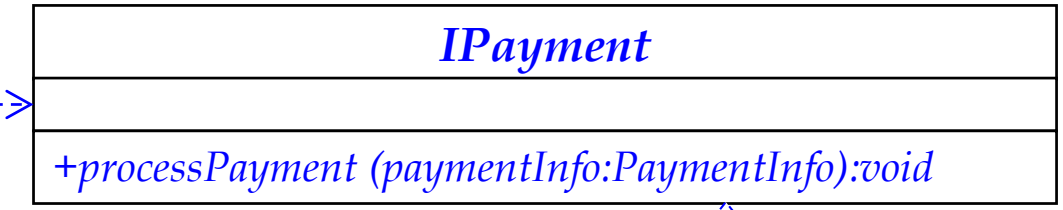
↓

说明 createOrder() 的方法内部必然对支付方式和物流跟踪存在依赖

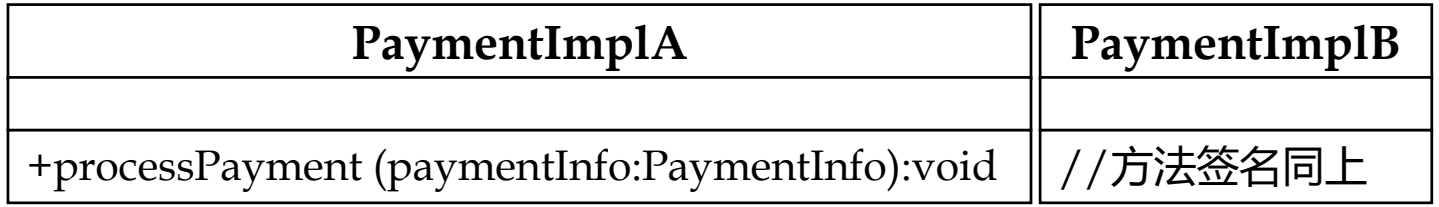


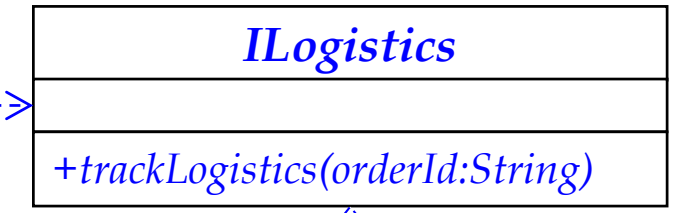
而且依赖频繁发生变化，因此应该对依赖的内容建立稳定的接口，对接口编程

在支付和物流变更比较频繁的情况下，可以重构到这种程度，从根本上解决 Order 创建问题

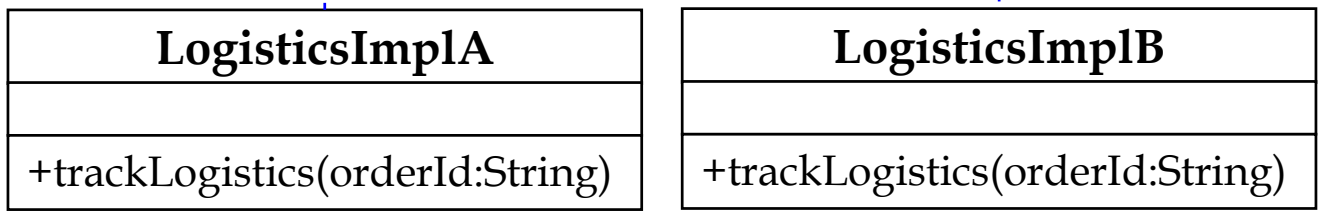


建立稳定的支付接口





建立稳定的物流接口



1.4 基于单一职责原则重构练习 - 代码深度重构

```java
public interface ILogistics {
    void trackLogistics(String orderId);
}
```

```java
public interface IPayment {
    void processPayment(PaymentInfo paymentInfo);
}
```

```java
public class LogisticsImplA implements ILogistics{
    @Override
    public void trackLogistics(String orderId) {
        // 具体的物流跟踪代码
     }
}
```

```java
public class PaymentImplA implements IPayment{
    @Override
    public void processPayment(PaymentInfo paymentInfo) {
        // 具体支付方式处理
     }
}
```

2.开闭原则

Open Closed Principle，OCP

2.1 开闭原则的定义

- Software entities (modules, classes, functions, etc.) should be open for extension ,but closed for modification 软件实体（模块、类、方法等）应该“对扩展开放、对修改关闭”。
  - 开闭原则是面向对象设计中，最基础的设计原则，是所有面向对象原则的核心，是高质量设计的判断标准。
  - 添加一个新的功能应该是，在已有代码基础上扩展代码（新增模块、类、方法等），而非修改已有代码（修改模块、类、方法等）。

2.2 不符合开闭原则的例子：通过修改已有代码实现功能扩展

对当前代码新增需求：添加一个功能，当每秒钟接口超时请求个数超过阈值触发告警发送通知

```
public class Alert { //微服务接口调用异常报警类
    private AlertRule rule;
    private Notification notification;

    public Alert(AlertRule rule, Notification notification) {
        this.rule = rule;
        this.notification = notification;
    }

    // check方法职责过多，不符合单一职责 SRP，本例重点不在 SRP，为了说明 OCP，后续优化重构
    public void check(String api, long requestCount, long errorCount, long durationOfSeconds) {
        long tps = requestCount / durationOfSeconds;
        if (tps > rule.getMatchedRule(api).getMaxTps()) {// tps超阈值发警告
            notification.notify(NotificationEmergencyLevel.URGENCY, "...");
        }
        if (errorCount > rule.getMatchedRule(api).getMaxErrorCount()) { // 请求出错数超阈值发警告
            notification.notify(NotificationEmergencyLevel.SEVERE, "...");
        }
    }
}
```

2.2 不符合开闭原则的例子：通过修改已有代码实现功能扩展

```
public class Alert {
    // ... 省略 AlertRule/Notification 属性和构造函数...

    // 改动一：添加参数 timeoutCount
    public void check(String api, long requestCount, long errorCount, long durationOfSeconds,
                                                                    long timeoutCount) {
        long tps = requestCount / durationOfSeconds;
        if (tps > rule.getMatchedRule(api).getMaxTps()) {
            notification.notify(NotificationEmergencyLevel.URGENCY, "...");
        }
        if (errorCount > rule.getMatchedRule(api).getMaxErrorCount()) {
            notification.notify(NotificationEmergencyLevel.SEVERE, "...");
        }
        // 改动二：添加接口超时处理逻辑
        long timeoutTps = timeoutCount / durationOfSeconds;
        if (timeoutTps > rule.getMatchedRule(api).getMaxTimeoutTps()) {
            notification.notify(NotificationEmergencyLevel.URGENCY, "...");
        }
    }
}
```

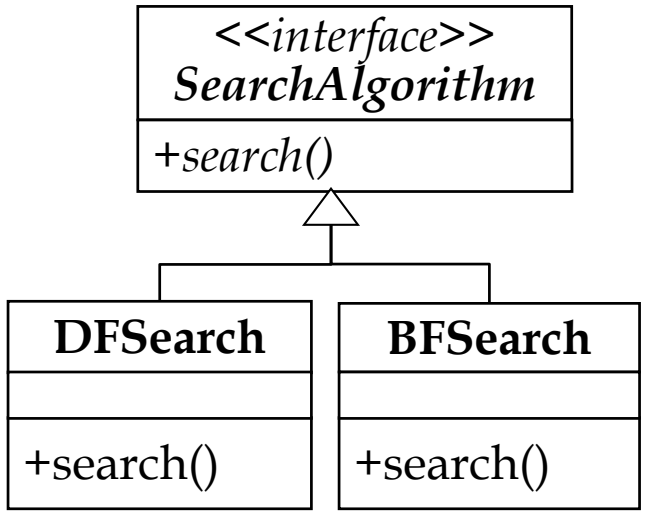


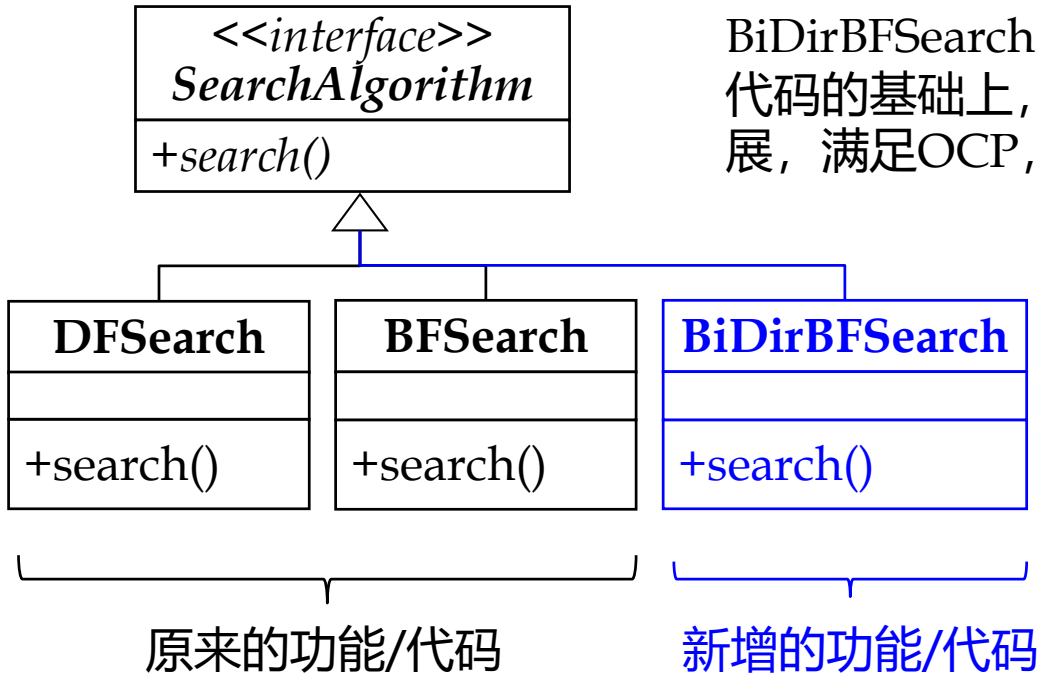
不满足OCP

显然，为了添加超时请求个数超过阈值触发告警发送通知这一需求，需要改动本类代码

2.3 符合开闭原则的例子

新增的功能/代码时





BiDirBFSearch 类在不修改原有的代码的基础上，原有代码进行了扩展，满足OCP，是一个良好的设计

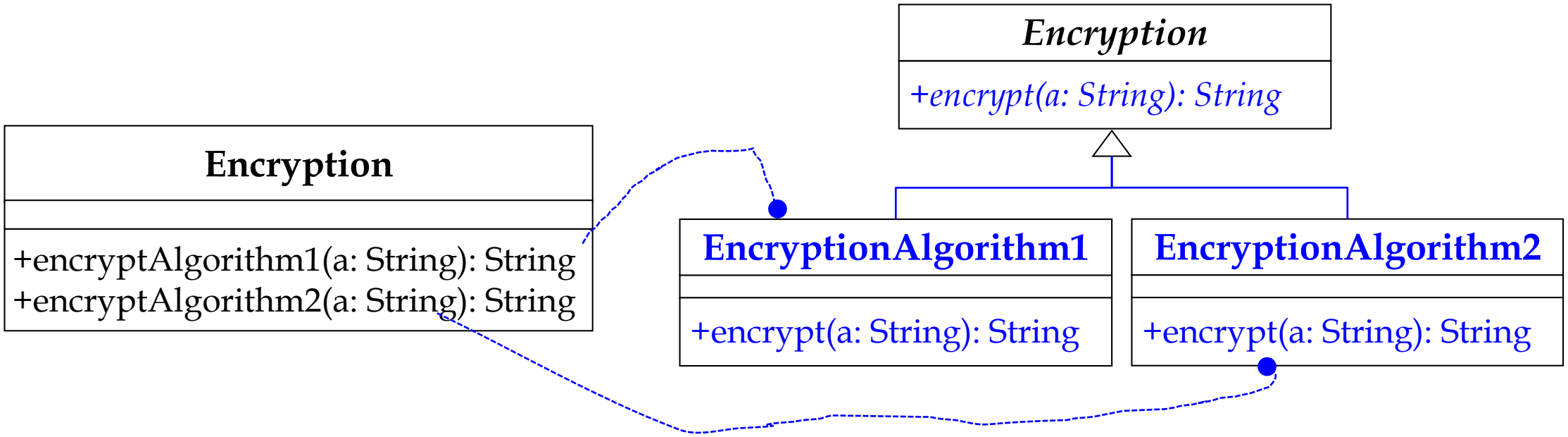


满足OCP

2.4 OCP 练习1：设计重构，要求满足开闭原则

- 例：设计一个程序对字符串进行加密。目前有两种不同的加密算法：encryptionAlgorithm1 和 encryptionAlgorithm2，设计如下，请重构。

统一算法的名字为 encrypt，并用接口类进行封装



每一种加密算法变成了实现类：可以任意添加实现类且不修改原有代码，满足开闭原则 OCP

2.4 OCP 练习2：设计重构 - 要求满足开闭原则

```
public class Alert {
    // ... 省略 AlertRule/Notification 属性和构造函数...

    // 改动一：添加参数 timeoutCount
    public void check(String api, long requestCount, long errorCount, long durationOfSeconds,
                                                                    long timeoutCount) {
        long tps = requestCount / durationOfSeconds;
        if (tps > rule.getMatchedRule(api).getMaxTps()) {
            notification.notify(NotificationEmergencyLevel.URGENCY, "...");
        }
        if (errorCount > rule.getMatchedRule(api).getMaxErrorCount()) {
            notification.notify(NotificationEmergencyLevel.SEVERE, "...");
        }
        // 改动二：添加接口超时处理逻辑
        long timeoutTps = timeoutCount / durationOfSeconds;
        if (timeoutTps > rule.getMatchedRule(api).getMaxTimeoutTps()) {
            notification.notify(NotificationEmergencyLevel.URGENCY, "...");
        }
    }
}
```



不满足OCP

显然，为了添加超时请求个数超过阈值触发告警发送通知这一需求，需要改动本类代码

2.4 OCP 练习2：设计重构 – 问题分析

```
public class Alert { //微服务接口调用异常报警类
    private AlertRule rule; //警告规则
    private Notification notification; // 通知方式

    public Alert(AlertRule rule, Notification notification) {
        this.rule = rule;
        this.notification = notification;
    }

    public void check(String api, long requestCount, long errorCount, long durationOfSeconds) {

        long tps = requestCount / durationOfSeconds;
        if (tps > rule.getMatchedRule(api).getMaxTps()) {
            notification.notify(NotificationEmergencyLevel.URGENCY, “...”);
        }
        if (errorCount > rule.getMatchedRule(api).getMaxErrorCount()) {
            notification.notify(NotificationEmergencyLevel.SEVERE, "...");
        }
    }
}
```

问题1：check方法职责过多

↓

能否将职责分离?

问题2：参数过长

↓

能否将参数封装成一个参数类?

2.4 OCP 练习2：设计重构 – 参数封装

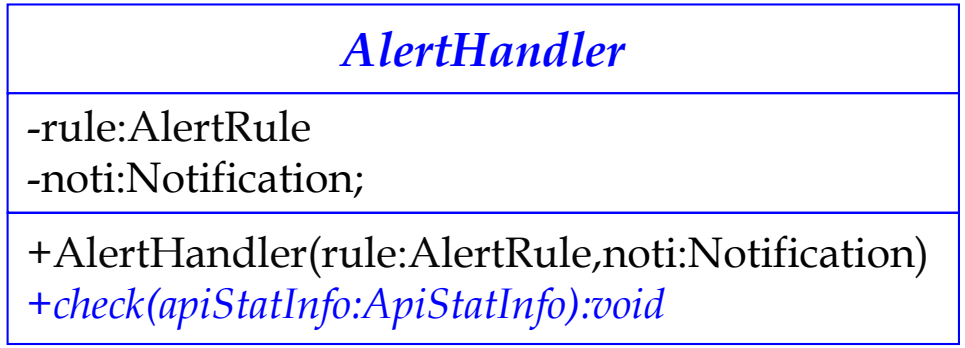
先解决问题2，参数过多的问题。将封装成一个类

```
public class ApiStatInfo {
    private String api;
    private long requestCount;
    private long errorCount;
    private long durationOfSeconds;
    // 无参构造
     public ApiStatInfo() {
    }
    //全参构造
     public ApiStatInfo(String api, long requestCount, long errorCount, long durationOfSeconds) {
        this.api = api;
        this.requestCount = requestCount;
        this.errorCount = errorCount;
        this.durationOfSeconds = durationOfSeconds;
        this.timeoutCount = timeoutCount;
    }
    // 省略setter/getter
}
```

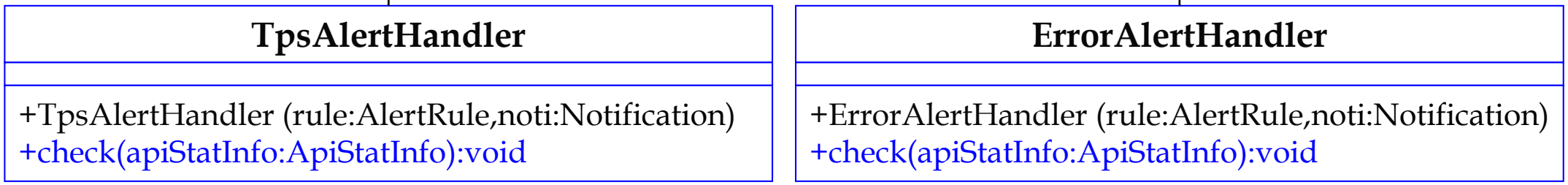
2.4 OCP 练习2：设计重构 – 分离职责

接下来解决问题1，分离职责

首先抽象出警报处理超类



其次在子类中封装各类型的处理逻辑

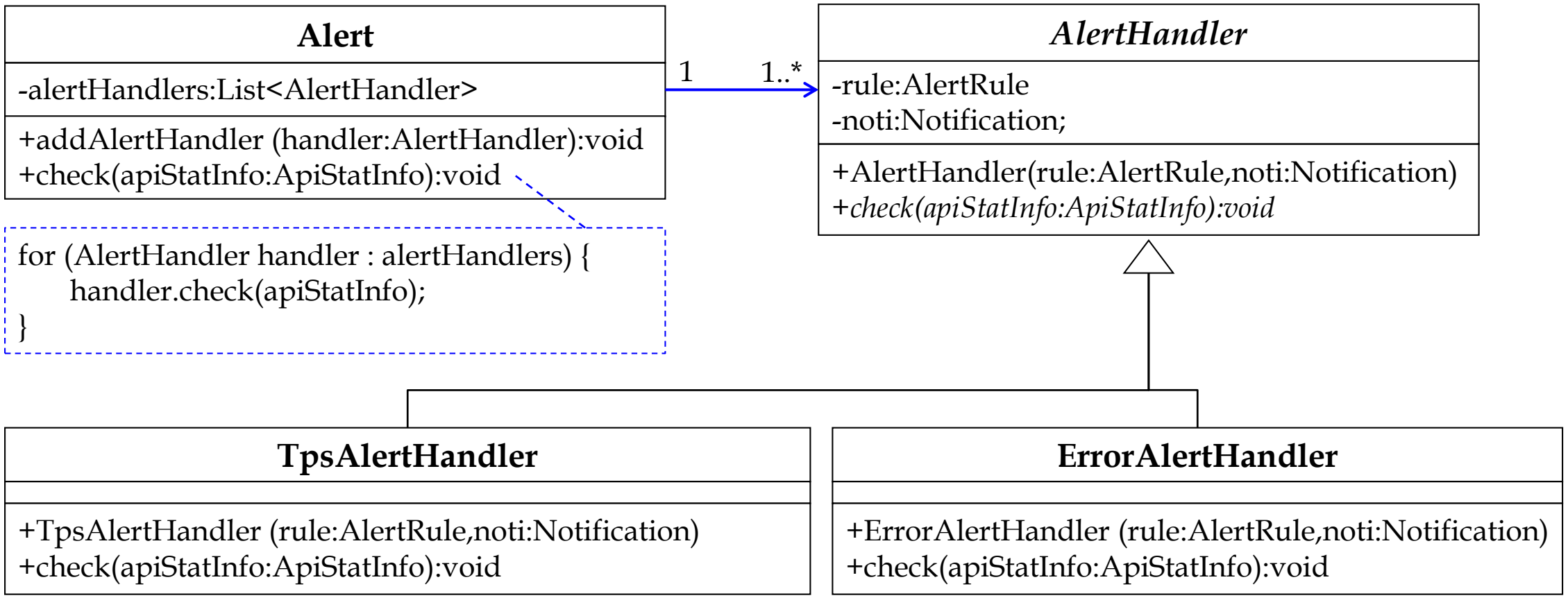


在check中处理 Tps 超阈值的处理逻辑

在check中处理 Error 超阈值的处理逻辑

这样对警报的处理逻辑就分离出来了。但是如何用这些处理逻辑?

2.4 OCP 练习2：设计重构 – Alert 类使用警报处理



Alert 中设置一个容器 ArrayList<AlertHandler>，存放各种 AlertHandler 的具体对象。而其check 方法，通过遍历的方式取出各个 AlertHandler对象，执行 AlertHandler 对象本身的检查即可。

2.4 OCP 练习2：设计重构 – 代码实现

```
public abstract class AlertHandler {
    protected AlertRule rule;
    protected Notification notification;
    public AlertHandler(AlertRule rule, Notification notification) {
        this.rule = rule;
        this.notification = notification;
    }
    public abstract void check(ApiStatInfo apiStatInfo);
}
```

```
public class ErrorAlertHandler extends AlertHandler {
    public ErrorAlertHandler(AlertRule rule, Notification notification) {
        super(rule, notification);
    }
    @Override
    public void check(ApiStatInfo apiStatInfo) {
        if (apiStatInfo.getErrorCount() > rule.getMatchedRule(apiStatInfo.getApi())
                                                              .getMaxErrorCount()) {
            notification.notify(NotificationEmergencyLevel.SEVERE, "...");
        }
    }
}
```

2.4 OCP 练习2：设计重构 – 代码实现

```
public class Demo {
    AlertRule alertRule = new AlertRule(/*. 省略参数.*/);
    Notification notification = new Notification(/*. 省略参数.*/);

    Alert alert = new Alert();

    // 在客户端设置各类型的处理逻辑
    alert.addAlertHandler(new TpsAlertHandler(alertRule, notification));
    alert.addAlertHandler(new ErrorAlertHandler(alertRule, notification));

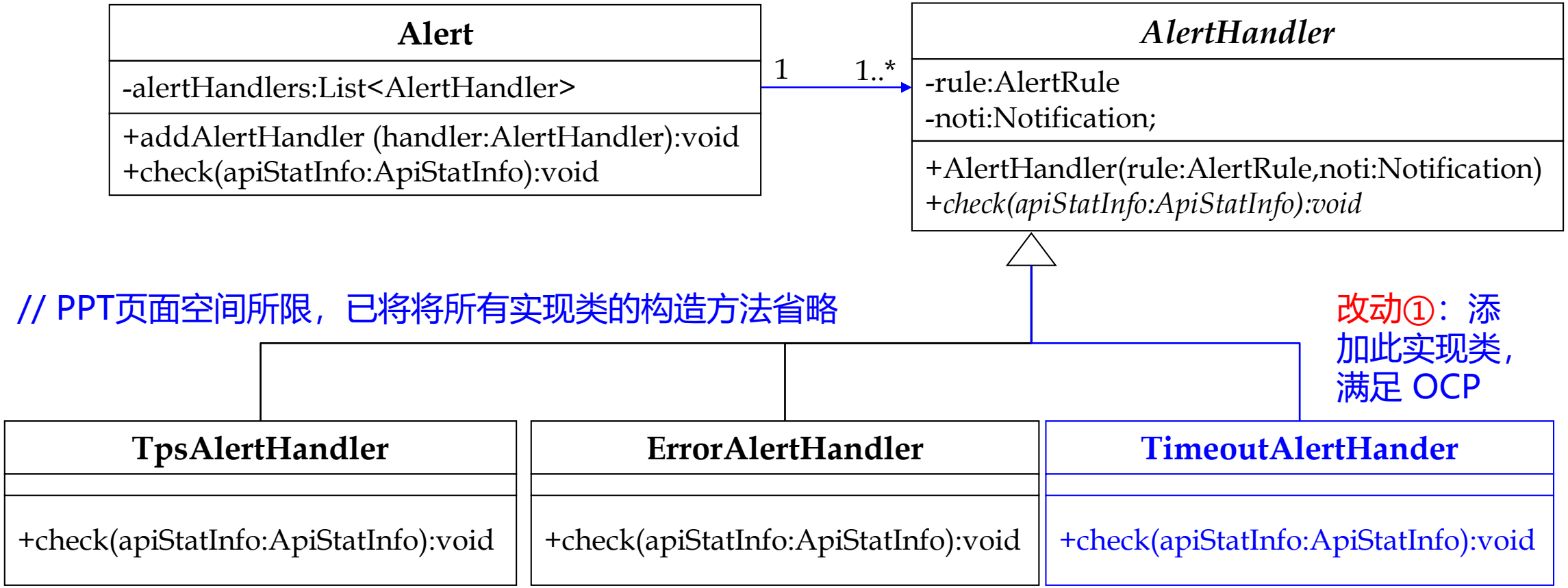
    ApiStatInfo apiStatInfo = new ApiStatInfo();

    // ... 省略设置 apiStatInfo 数据值的代码

    alert.check(apiStatInfo);
}
```

2.4 OCP 练习2：设计重构 – 设计分析

- 要求：添加一个功能，当每秒钟接口超时请求个数超过阈值触发告警发送通知



2.4 OCP 练习2：设计重构 – 设计分析

```
public class ApiStatInfo {
    private String api;
    private long requestCount;
    private long errorCount;
    private long durationOfSeconds;
    private long timeoutCount;
    // 无参构造
    public ApiStatInfo() {
    }
    //全参构造
    public ApiStatInfo(String api, long requestCount, long errorCount, long durationOfSeconds,
                       long timeoutCount) {
        this.api = api;
        this.requestCount = requestCount;
        this.errorCount = errorCount;
        this.durationOfSeconds = durationOfSeconds;
        this.timeoutCount = timeoutCount;
    }
    // 省略setter/getter
}
```

改动②：添加 timeoutCount 属性，对应修改构造函数和setter/getter

2.4 OCP 练习2：设计重构 – 设计分析

软件实体（模块、类、方法等）应该“对扩展开放、对修改关闭”。在粗代码粒度下，被认定为“修改”，在细代码粒度下，又可以被认定为“扩展”：只要它没有破坏原有的代码的正常运行，没有破坏原有的单元测试，就可以说，这是一个合格的代码改动。通常，我们从粗粒度角度判断是否满足开闭原则OCP

```
    private long durationOfSeconds;
    private long timeoutCount;
    // 无参构造
    public ApiStatInfo() {
    }
    //全参构造
    public ApiStatInfo(String api, long requestCount, long errorCount, long durationOfSeconds,
                                                                    long timeoutCount) {
        this.api = api;
        this.requestCount = requestCount;
        this.errorCount = errorCount;
        this.durationOfSeconds = durationOfSeconds;
        this.timeoutCount = timeoutCount;
    }
    // 省略setter/getter
}
```

改动②：添加 timeoutCount 属性，对应修改构造函数和setter/getter

题外话：从粗粒度角度上看，这种设计显然不满足开闭原则OCP。如何设计才能满足OCP?

2.4 OCP 练习2：设计重构 – 设计分析

```
public class Demo {
    AlertRule alertRule = new AlertRule(/*. 省略参数.*/);
    Notification notification = new Notification(/*. 省略参数.*/);

    Alert alert = new Alert();

    alert.addAlertHandler(new TpsAlertHandler(alertRule, notification));
    alert.addAlertHandler(new ErrorAlertHandler(alertRule, notification));
    alert.addAlertHandler(new TimeoutAlertHandler (alertRule, notification));

    ApiStatInfo apiStatInfo = new ApiStatInfo();

    // ... 省略设置 apiStatInfo 数据值的代码
    apiStatInfo.setTimeoutCount(289);

    alert.check(apiStatInfo);
}
```

改动③：需要往Alert的list中添加 timeoutAlertHandle

改动④：设置 tiemoutCount 值

改动③④属于客户端Demo的内部，类需要创建、组装、并且做一些初始化操作，才能构建成可运行的的程序，这部分代码的修改在所难免

2.5 OCP 小结

- 写出支持“对扩展开放、对修改关闭”的代码的关键是预留扩展点。
- “唯一不变的只有变化本身”。不可能识别出所有的扩展点，即便你能识别出所有的扩展点，为这些地方都预留扩展点，这样做的成本也是不可接受的。我们没必要为一些遥远的、不一定发生的需求去提前买单，做过度设计。
- 有些情况下，代码的扩展性会跟可读性相冲突。Alert 告警的例子中，为了更好地支持扩展性，我们进行了重构，重构之后的代码要比之前的代码复杂，理解起来也更加有难度。

3. 里氏代换原则

Liskov Substitution Principle，LSP

3.1 里氏代换原则 Liskov Substitution Principle 的表述

- 这个原则最早是在 1986 年由 Barbara Liskov 提出，她是这么描述这条原则的：

> If S is a subtype of T, then objects of type T may be replaced with objects of type S, without breaking the program。
> 若对每个类型 S 的对象 o1，都存在一个类型 T 的对象 o2，使得在所有针对 T 编写的程序 P 中，用 o1 替换 o2 后，程序 P 的行为不变，则 S 是 T 的子类型

- 1996 年，Robert Martin 在他的 SOLID 原则中，重新描述了这个原则：

> Functions that use pointers of references to base classes must be able to use objects of derived classes without knowing it。
> 使用基类引用指针的函数必须能够在不知情的情况下使用派生类的对象

- 综合两者的描述，将里氏代换原则描述为：

> 子类对象（object of subtype/derived class）能够替换程序（program）中父类对象（object of base/parent class）出现的任何地方，并且保证原来程序的逻辑行为（behavior）不变及正确性不被破坏。

3.2 如何理解里氏代换原则 Liskov Substitution Principle

- 例如：

```
public class Transporter {
    private HttpClient httpClient;
    public Transporter(HttpClient httpClient) {   this.httpClient = httpClient;  }

    public Response sendRequest(Request request) {
        // ...use httpClient to send request
    }
}
```

父类使用 org.apache.http 库中的 HttpClient 类来传输网络数据

3.2 如何理解里氏代换原则 Liskov Substitution Principle

```
public class SecurityTransporter extends Transporter {
    private String appId;
    private String appToken;
    public SecurityTransporter(HttpClient httpClient, String appId, String appToken) {
        super(httpClient);
        this.appId = appId;
        this.appToken = appToken;
    }
    @Override
    public Response sendRequest(Request request) {
        if (StringUtils.isNotBlank(appId) && StringUtils.isNotBlank(appToken)) {
            request.addPayload("app-id", appId);
            request.addPayload("app-token", appToken);
        }
        return super.sendRequest(request);
    }
}
```

子类增加了额外的功能，支持传输 appId 和 appToken 安全认证信息

3.2 如何理解里氏代换原则 Liskov Substitution Principle

```
public class ApplicationContext {
    public void demoFunction(Transporter transporter) {
        Reuqest request = new Request();
        //... 省略设置 request 中数据值的代码...
        Response response = transporter.sendRequest(request);
        //... 省略其他逻辑...
    }
}
```

子类SecurityTransporter对象替换父类Transporter对象后，在ApplicationContext 的逻辑没有影响

```
public class TestApplicationContext {
    public static void main(String[] args) {
        // 里式替换原则
        ApplicationContext context = new ApplicationContext();
        //子类对象替换父类对象
        context.demofunction(new SecurityTransporter(/* 省略参数 */));
    }
}
```

说明子类 SecurityTransporter 的设计完全符合里式替换原则。接下来改造一下 该类

3.2 如何理解里氏代换原则 Liskov Substitution Principle

```
public class Transporter {
    private HttpClient httpClient;
    public Transporter(HttpClient httpClient) {   this.httpClient = httpClient;  }
    public Response sendRequest(Request request) {
        // ...use httpClient to send request
    }
}
```

```
// 对子类进行改造，子类的方法会抛异常；注意，父类中并不抛异常：
public class SecurityTransporter extends Transporter {
    //... 省略其他代码..
    @Override
    public Response sendRequest(Request request) {
        if (StringUtils.isBlank(appId) || StringUtils.isBlank(appToken)) {
            throw new NoAuthorizationRuntimeException(...);
        }
        request.addPayload("app-id", appId);
        request.addPayload("app-token", appToken);
        return super.sendRequest(request);
    }
}
```

3.2 如何理解里氏代换原则 Liskov Substitution Principle

```
public class Transporter {
    private HttpClient httpClient;
    public Transporter(HttpClient httpClient) {   this.httpClient = httpClient;  }
    public Response sendRequest(Request request) {
        // ...use httpClient to send request                    //代码不抛异常
    }
}
```

```
// 对子类进行改造，子类的方法会抛异常；注意，父类中并不抛异常：
public class SecurityTransporter extends Transporter {
    //... 省略其他代码..
    @Override
    public Response sendRequest(Request request) {
        if (StringUtils.isBlank(appId) || StringUtils.isBlank(appToken)) {
            throw new NoAuthorizationRuntimeException(...);          //代码抛异常
        }
        request.addPayload("app-id", appId);
        request.addPayload("app-token", appToken);
```

不满足里氏代换原则

尽管代码中抛出的是运行时异常（Runtime Exception），可以不在代码中显式地捕获处理，但子类替换父类传递进Application类的demoFunction函数之后，整个程序的逻辑行为有了改变：由不抛异常变成抛异常，因此不满足里氏代换原则

3.3 里氏代换原则是对多态的一种更严格的限制

- 从定义描述和代码实现上来看，多态和里式替换有点类似，但它们关注的角度不一样：
  - 多态是面向对象编程的一大特性，也是面向对象编程语言的一种语法。它是一种代码实现的思路。
  - 里式替换是一种设计原则，是用来指导继承关系中子类该如何设计的，子类的设计要保证在替换父类的时候，不改变原有程序的逻辑以及不破坏原有程序的正确性。
- 里氏代换原则是多态基础上提出的一种原则。换言之，代码是多态的，不一定符合里氏代换原则；代码符合里氏代换原则，必然是多态的。

3.4 里式替换原则更加能落地的描述

- Design By Contract。即：按照协议来设计。子类在设计的时候，要遵守父类的行为约定（或者叫协议）。父类定义了函数的行为约定，那子类可以改变函数的内部实现逻辑，但不能改变函数原有的行为约定。
- 行为约定包括：函数声明要实现的功能；对输入、输出、异常的约定；甚至包括注释中所罗列的任何特殊说明。

3.5 反例：违反里氏代换原则举例-不按照协议来设计

- 例1：子类违背父类声明要实现的功能

父类中提供的 sortOrdersByAmount() 订单排序函数，是按照金额从小到大来给订单排序
子类重写这个 sortOrdersByAmount() 订单排序函数，是按照创建日期来给订单排序的。
子类的设计就违背里式替换原则。

3.5 反例：违反里氏代换原则举例-不按照协议来设计

- 例2：子类违背父类对输入、输出、异常的约定

父类中，某个函数约定：运行出错的时候返回 null；获取数据为空的时候返回空集合（empty collection）。
子类重载函数之后，实现变了，运行出错返回异常（exception），获取不到数据返回 null。
那子类的设计就违背里式替换原则。

父类中，某个函数约定，输入数据可以是任意整数，但子类实现的时候，只允许输入数据是正整数，负数就抛出，也就是说，子类对输入的数据的校验比父类更加严格，那子类的设计就违背了里式替换原则

父类中，某个函数约定，只会抛出 ArgumentNullException 异常，那子类的设计实现中只允许抛出 ArgumentNullException 异常，任何其他异常的抛出，都会导致子类违背里式替换原则。

3.5 反例：违反里氏代换原则举例-不按照协议来设计

- 例3：子类违背父类注释中所罗列的任何特殊说明

父类中定义的 withdraw() 提现函数的注释是这么写的： “用户的提现金额不得超过账户余额……”，
而子类重写 withdraw() 函数之后，针对 VIP 账号实现了透支提现的功能，也就是提现金额可以大于账户余额，
那这个子类的设计也是不符合里式替换原则的

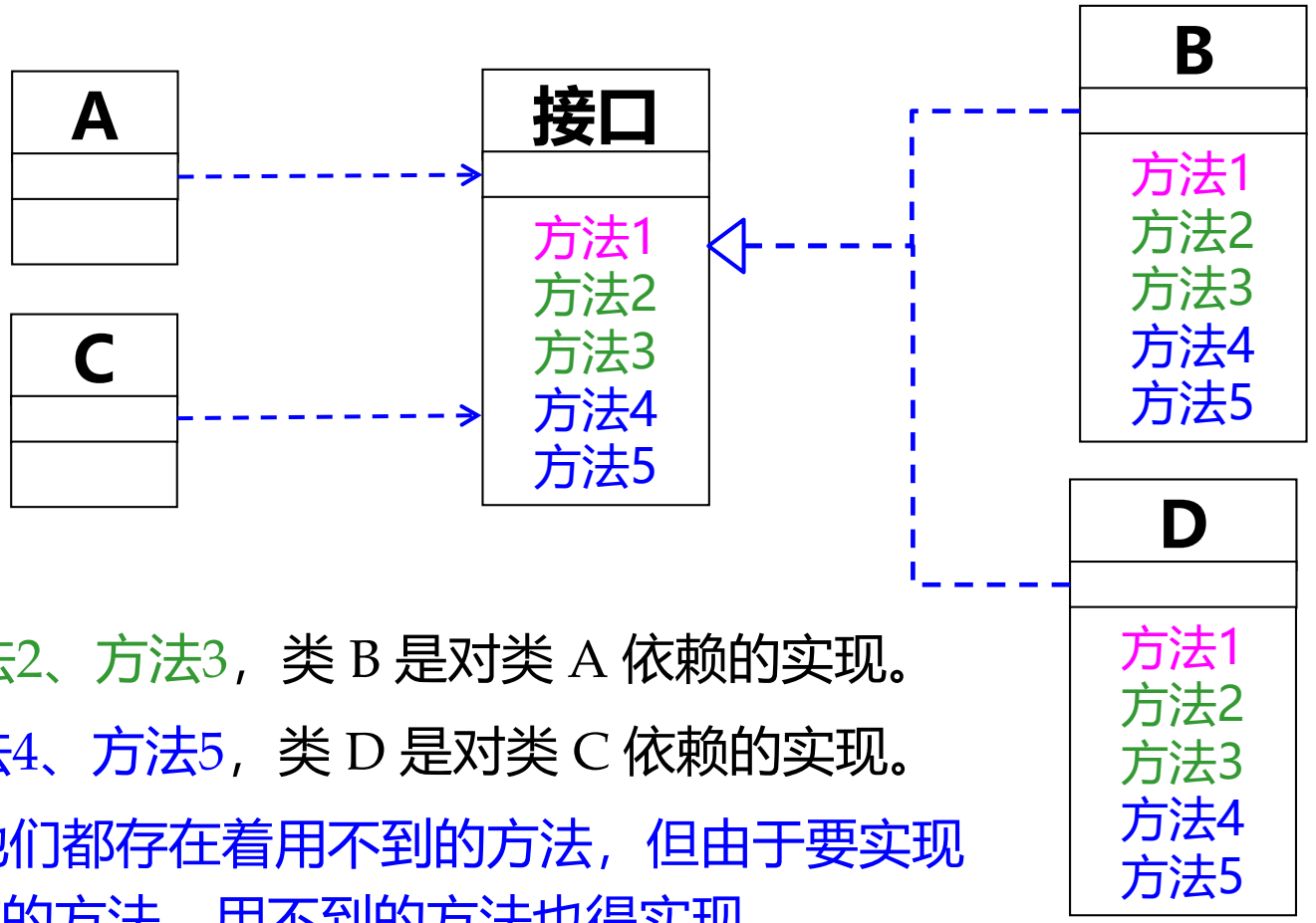
小窍门：那就是拿父类的单元测试去验证子类的代码。如果某些单元测试运行失败，就有可能说明，子类的设计实现没有完全地遵守父类的约定，子类有可能违背了里式替换原则。

4.接口隔离原则

Interface Segregation Principle，ISP

4.1 接口隔离原则解决的问题

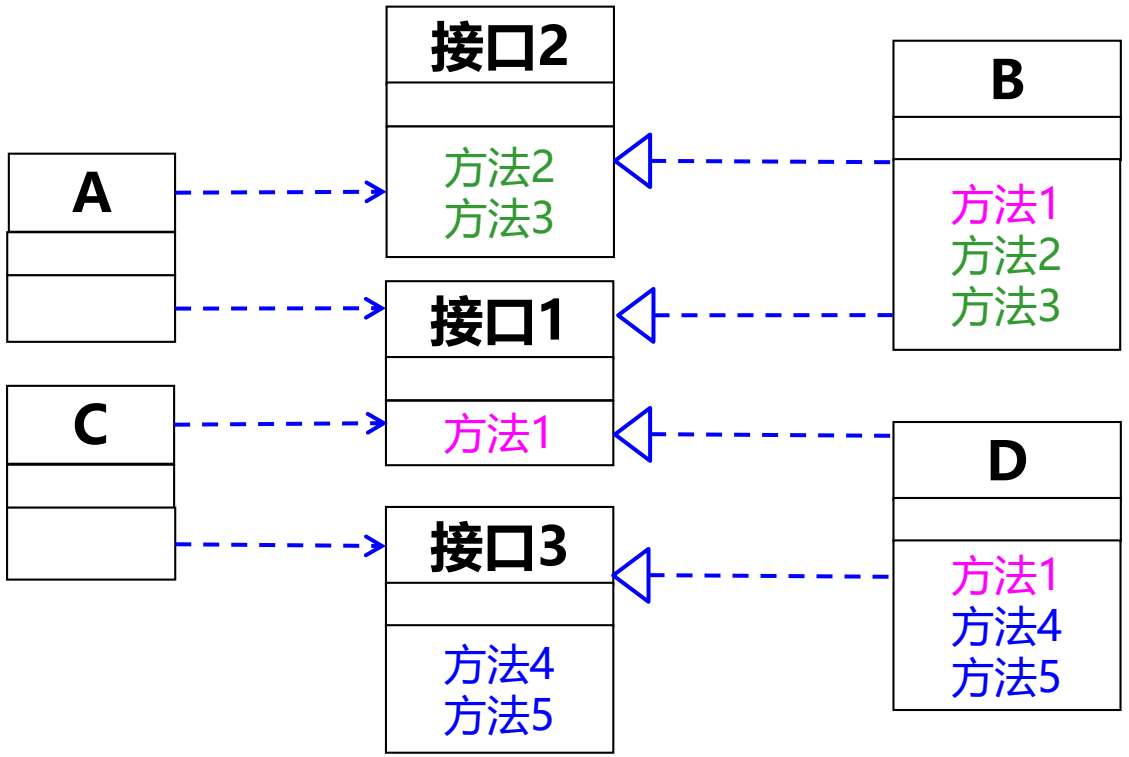
- 问题由来



- 类A依赖接口中的方法1、方法2、方法3，类 B 是对类 A 依赖的实现。
- 类C依赖接口中的方法1、方法4、方法5，类 D 是对类 C 依赖的实现。
- 对于类 B 和类 D来说，虽然他们都存在着用不到的方法，但由于要实现接口，就必须实现接口中所有的方法，用不到的方法也得实现。

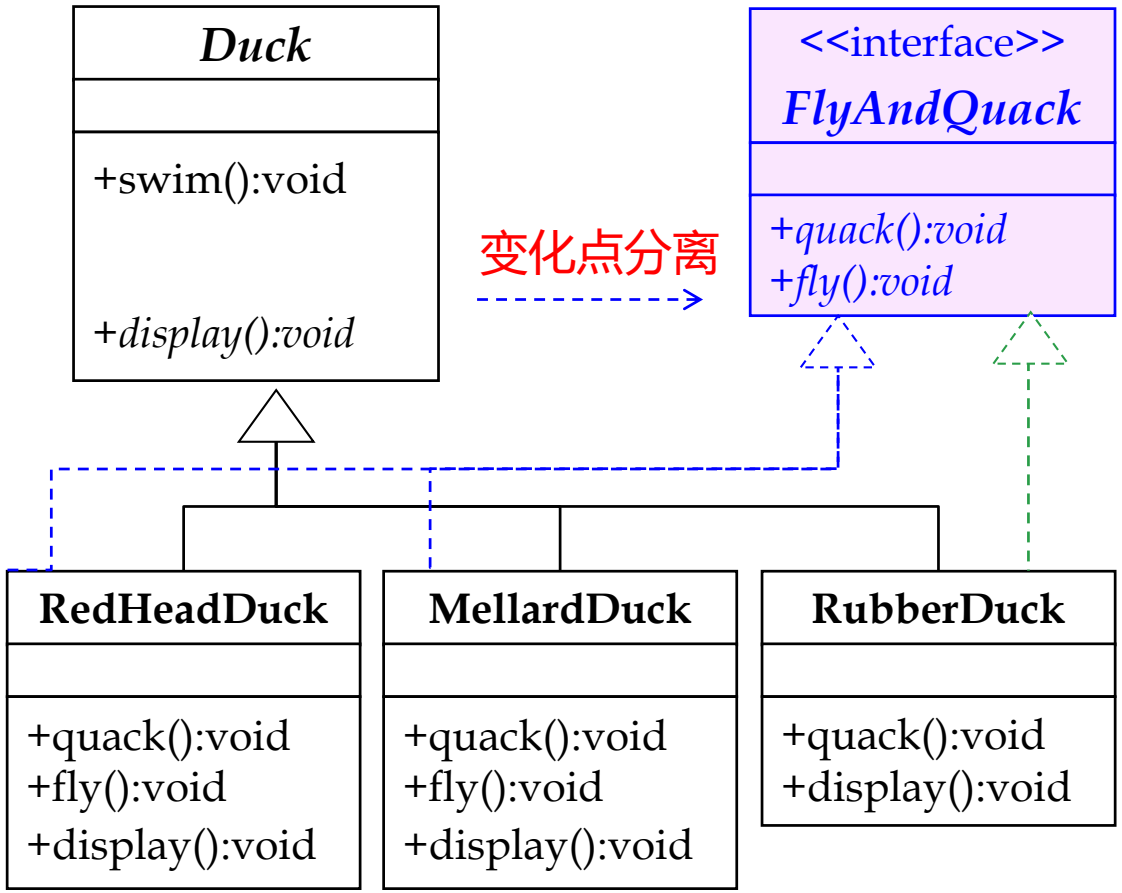
4.2 接口隔离原则的定义

- “Clients should not be forced to depend upon interfaces that they do not use。” 即：客户端不应该强迫依赖它不需要的接口。其中的“客户端”，可以理解为接口的调用者或者使用者



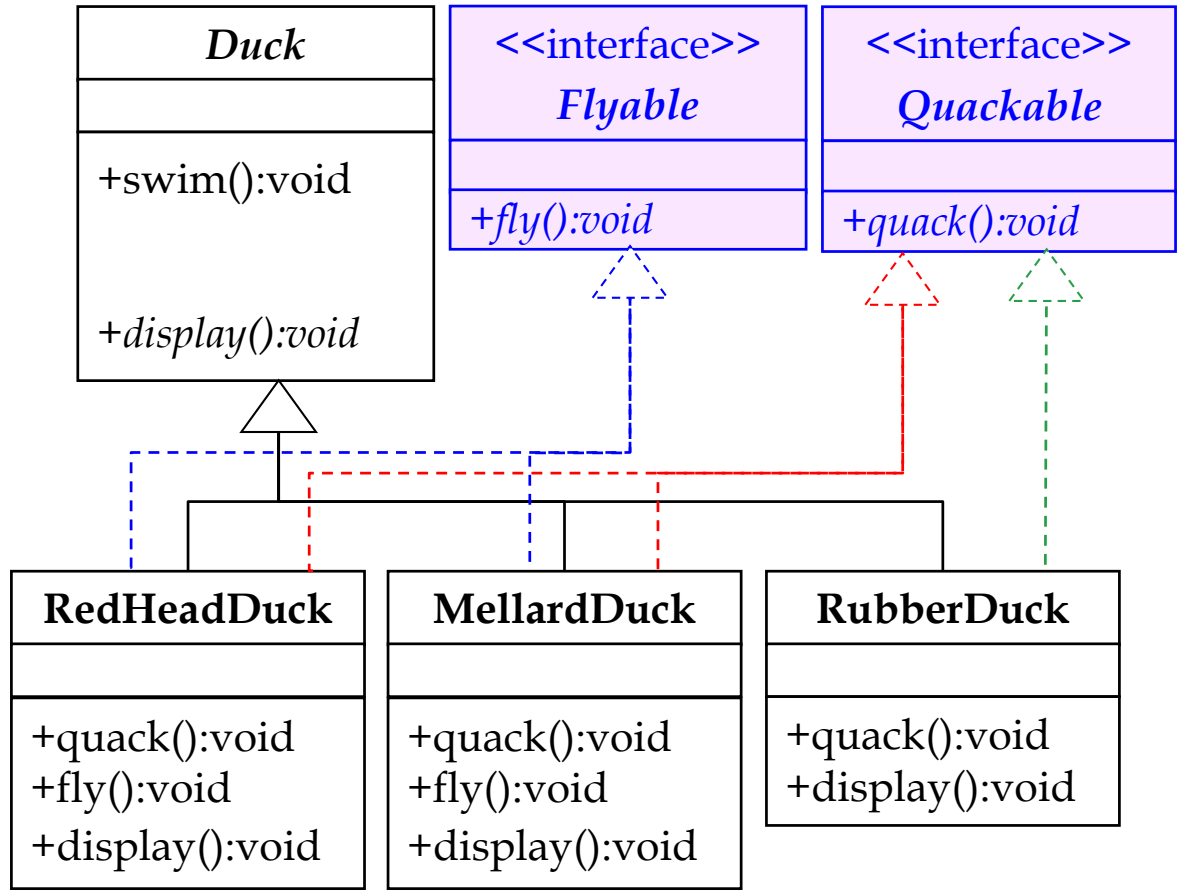
要为各个类建立专用的接口，而不要试图去建立一个很庞大的接口供所有依赖它的类去调用。

4.3 接口隔离原则的示例



方案1：quack() 和 fly() 分离到一个接口

实现 FlyAndQuack 接口类的子类必须 会飞 + 会叫，无论是不是具有这两种行为



方案2：quack() 和 fly() 分离到不同的接口

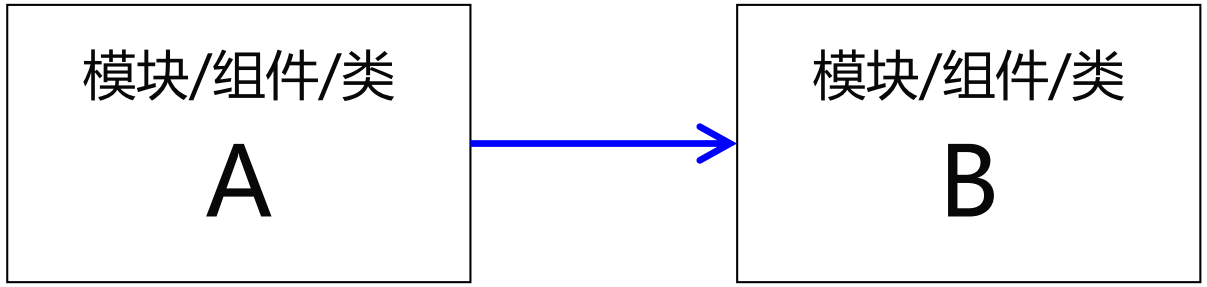
通过实现不同接口类满足：有的会飞，有的会叫，有的会飞+会叫；

5.依赖倒置原则

Dependence Inversion Principle，DIP

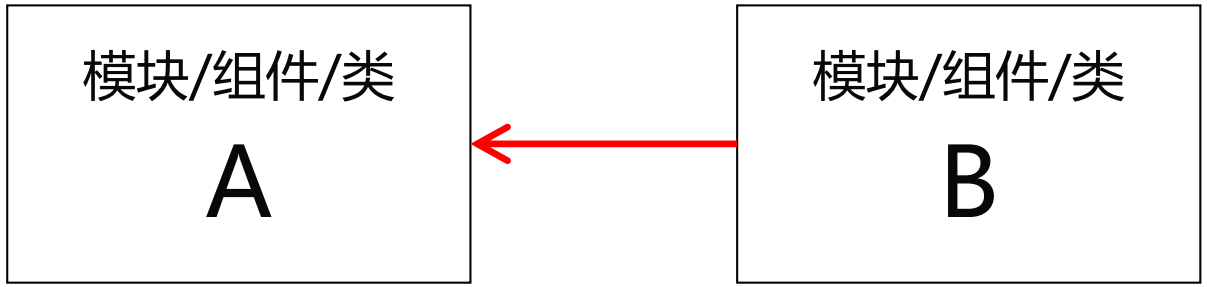
5.1 依赖倒置原则的核心思想

- 某个模块 A 中使用到另外一个模块 B 时，最直接的方式是在 A 中直接依赖 B



A 直接依赖于B，任何对 B 的更改都会直接影响到 A。

- 能否反其道而行之，模块 A 使用 模块 B 的时候，让 B 依赖 A?



变更为 B 依赖于 A，这样对 B 的更改不会直接影响到 A；而 A 的变更 影响到 B

- 所谓的“依赖倒置”又称为“依赖反转”，是指 A 和 B 的依赖方向（即依赖关系）的倒置

5.2 依赖倒置原则的表述

- Dependence Inversion Principle 中文称“依赖倒置原则” 或“依赖反转原则” 。定义原文：

High-level modules shouldn’t depend on low-level modules. Both modules should depend on abstractions. In addition, abstractions shouldn’t depend on details. Details depend on abstractions.

两个方面

①高层模块与低层模块的依赖关系：高层模块不应该依赖低层模块，两者都应该依赖于抽象

②抽象与细节的依赖关系：抽象不应该依赖于细节，细节应该取决于抽象

四个关键词

高层模块：调用者

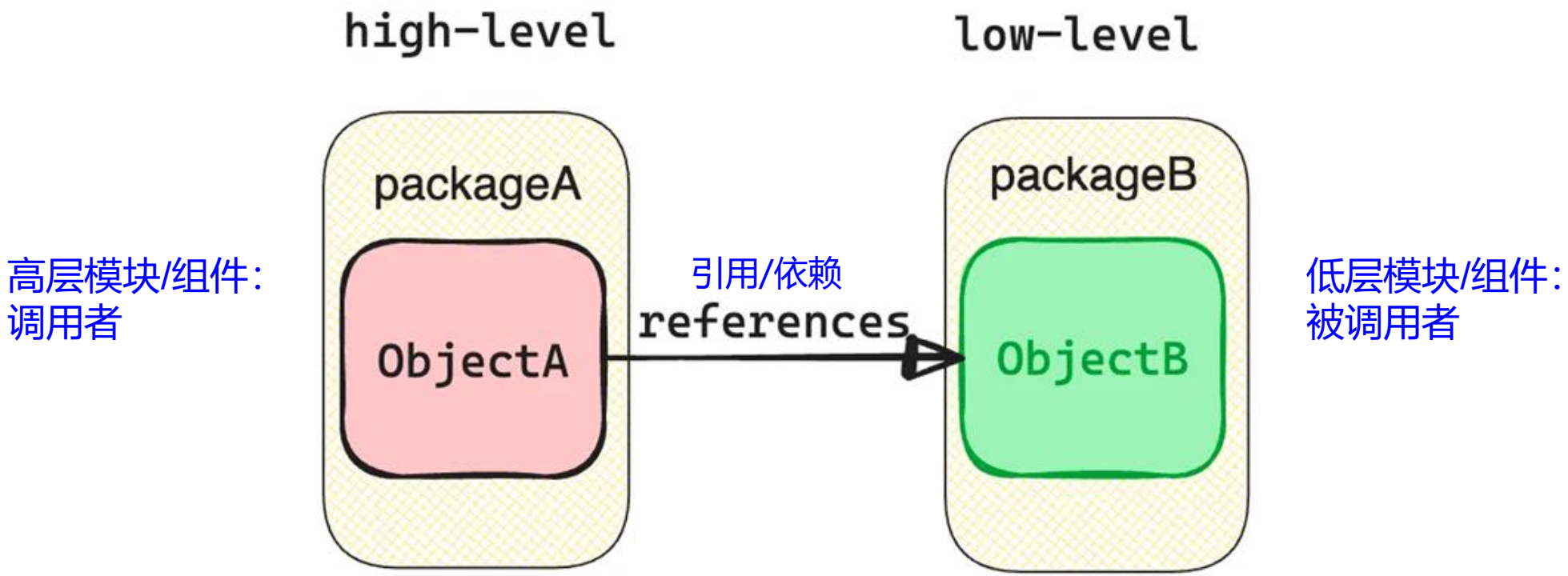
低层模块：被调用者

抽象：接口/抽象类

细节：具体类

5.3 依赖倒置原则的理解：直接依赖的问题

- 传统的直接依赖存在什么问题？如下为一张直接依赖的关系图：



高层组件 ObjectA 直接依赖于低层组件 ObjectB 去实现一些任务，但对于低层次组件的依赖限制了高层次组件被重用的可行性，因为任何对低层组件的更改都会直接影响高层组件。

5.3 依赖倒置原则的理解：直接依赖的问题

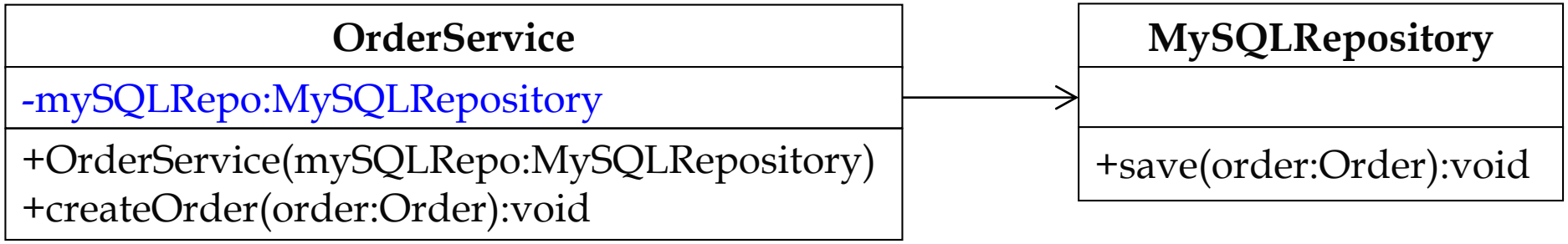
- 为了更好的说明直接依赖的问题，以一个真实的电商场景中订单处理为例

```
// 高层模块 OrderService，用于处理订单
public class OrderService {
    private MySQLOrderRepository mySQLRepository;
    public OrderService(MySQLRepository mySQLRepository) {
        this.mySQLRepository = mySQLRepository;
    }
    public void createOrder(Order order) {
        // 一些业务逻辑
        mySQLRepository.save(order);
    }
}
```

```
// 低层模块：MySQLRepository，用来存储和检索订单数据
public class MySQLRepository {
    public void save(Order order) {
        // 使用 MySQL数据库保存订单
     }
}
```

5.3 依赖倒置原则的理解：直接依赖的问题

- 上述代码中，OrderService直接依赖于 MySQLRepository。为了分析方便，绘制类图如下：



这种设计的缺点：

- ✓ 紧耦合：如果要把数据库从 MySQL切换到其他的数据库，我们需要修改 OrderService，因为它直接依赖于 MySQLRepository
- ✓ 难以测试：在进行单元测试时，我们无法轻松地对 OrderService 进行模拟，因为它直接依赖于具体实现 MySQLRepository。
- ✓ 重用性差：如果在另一个项目中我们需要使用 OrderService 但存储订单的方式不同，例如使用文件系统或远程服务，我们将无法直接重用 OrderService。

5.4 如何实现依赖倒置？- 方案一的基本思路

- 对于这些缺点，该如何解决？给出两种方案，方案一：

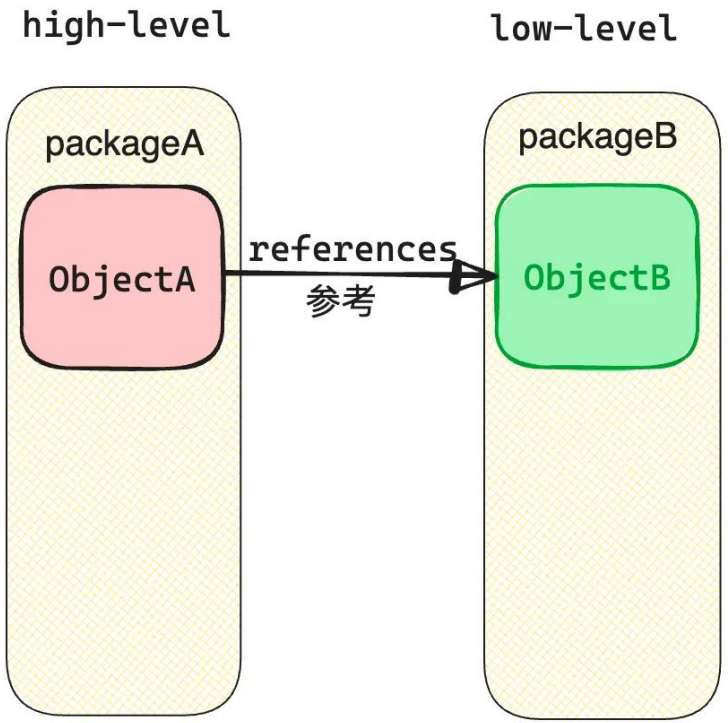
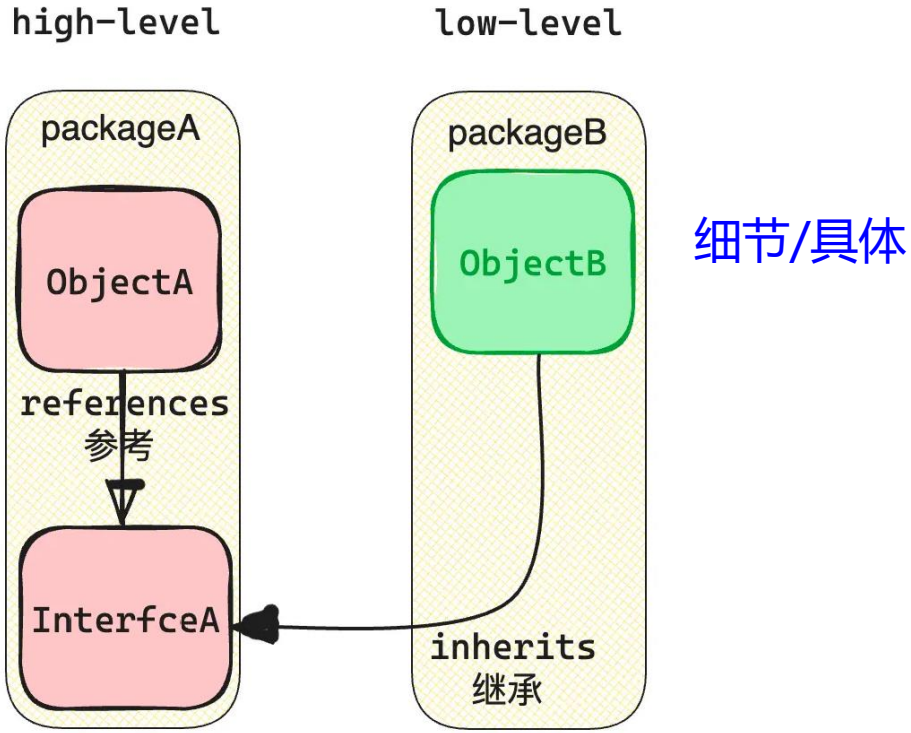


图1

改变为：

抽象



细节/具体

图2

图1-正向依赖：高层对象 A 依赖于底层对象 B 的实现；

图2-依赖倒置：把高层对象 A 对底层对象的需求抽象为一个接口 A，底层对象 B 实现了接口A，这就是依赖反转。

5.4 如何实现依赖倒置？- 方案一的具体设计

- 对应的，方案一的实现通过引入一个抽象层 OrderRepository来解耦高层模块和低层模块，整个关系图如下：

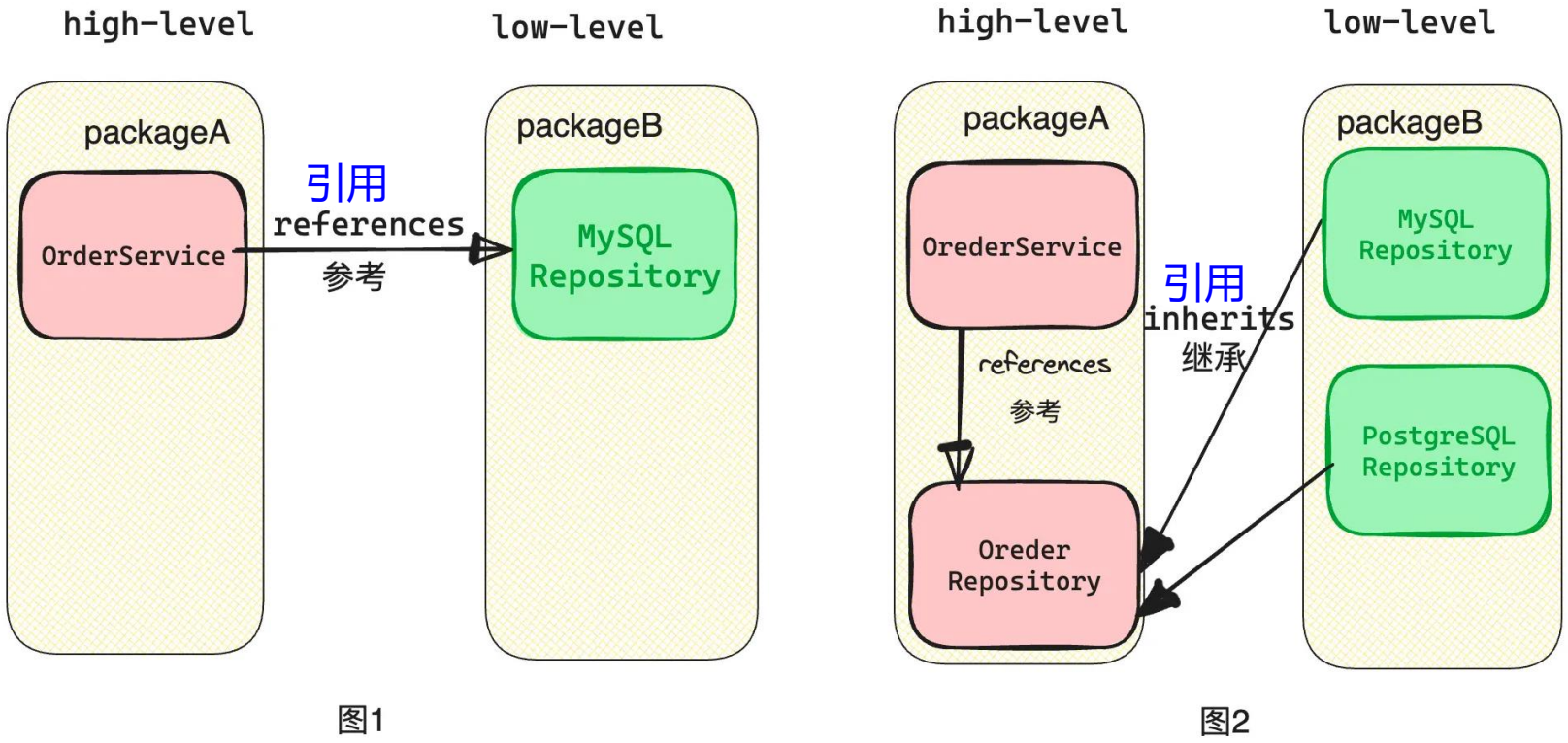
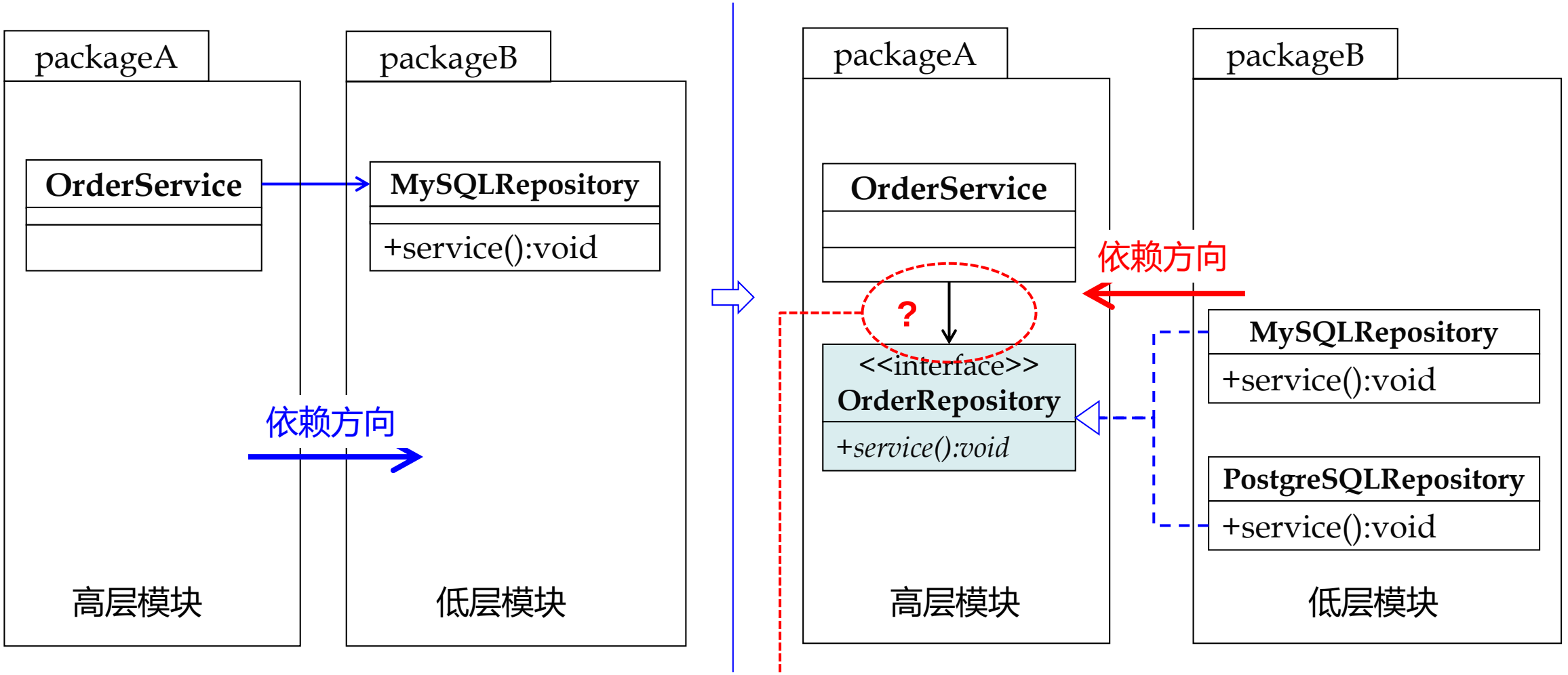


图1　　图2

OrderService依赖于 OrderRepository接口 而不是具体实现 MySQLRepository。这样，我们可以轻松替换低层实现而无需修改高层模块

5.4 如何实现依赖倒置？- 方案一的理解



红色椭圆框疑问点：抽象接口划分到高层模块是理解依赖倒置的关键。依赖导致是判断高层模块和底层模块的关系是否倒置！

5.4 如何实现依赖倒置？- 方案一的代码实现

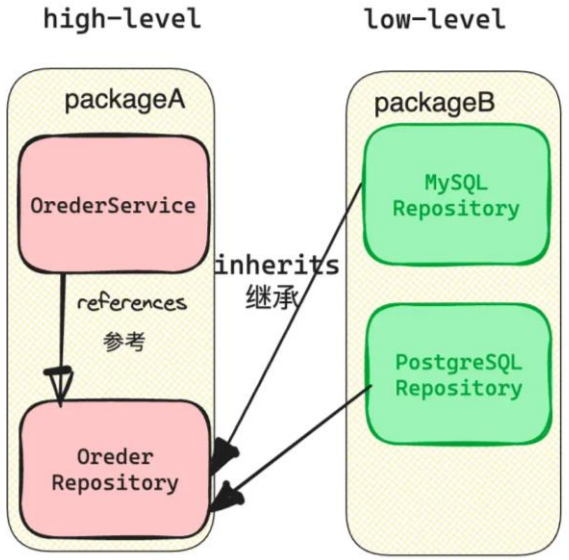
```java
// 抽象层：OrderRepository接口
public interface OrderRepository {
    void save(Order order);
}
```

```java
// 低层模块：MySQLRepository
public class MySQLRepository implements OrderRepository {
    @Override
    public void save(Order order) {
        // 使用 MySQL数据库保存订单
    }
}
```

```java
// 低层模块：PostgreSQLRepository
public class PostgreSQLRepository implements OrderRepository{
    @Override
    public void save(Order order) {
        // 使用 PostgreSQL 数据库保存订单
    }
}
```

5.4 如何实现依赖倒置？- 方案一的代码实现

```java
// 高层模块：OrderService
public class OrderService {
    private OrderRepository orderRepository;
    public OrderService(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
    }
    public void createOrder(Order order) {
        // 一些业务逻辑的代码省略
         orderRepository.save(order);
    }
}
```




```java
//测试类OrderServiceTest
public class OrderServiceTest {
    @Test
    public void testCreateOrder() {
        OrderRepository orderRepository = new MySQLRepository();
        //orderRepository = new PostgreSQLRepository();
        OrderService orderService = new OrderService(orderRepository);
    }
}
```

OrderService 变得更具重用性、可测试性更强，并且与具体的存储实现解耦，满足依赖倒置原则的要求

5.5 如何实现依赖倒置？- 方案二的基本思路

- 尽管方案一实现了依赖倒置，但是这种实现方式高层组件以及抽象组件是封装在一个包中，对低层组件的重用会差一些。

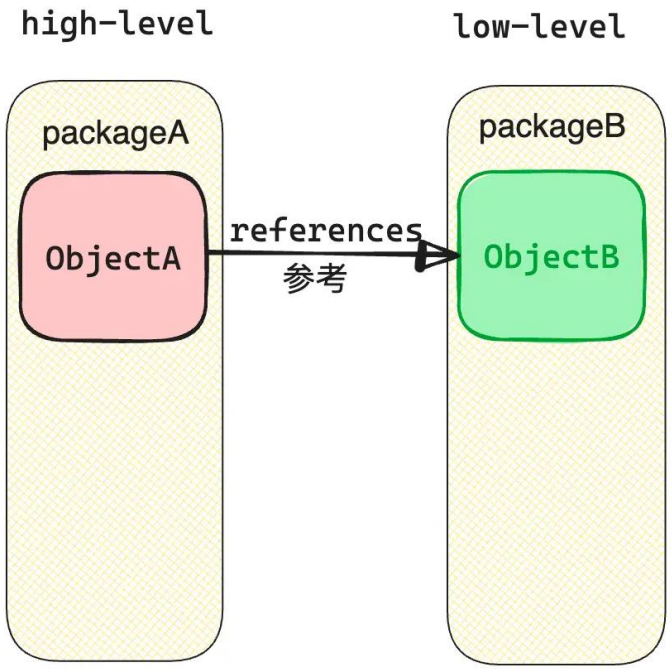


图1

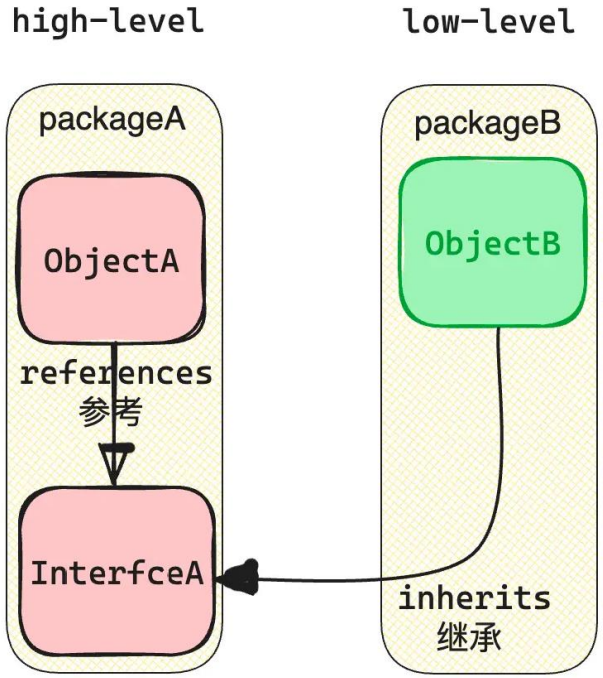
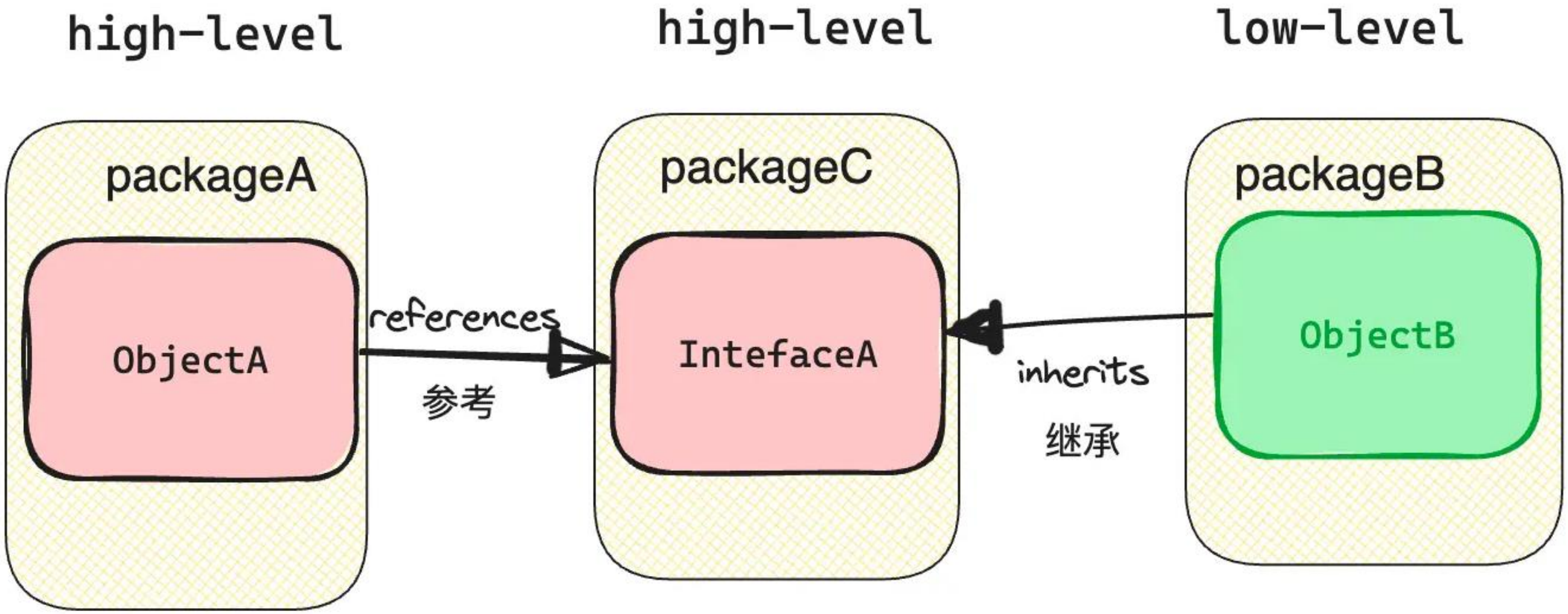


图2

为何说方案一对低层组件的重用会差一些：在面向对象系统设计中，如果重用了一个包中的某个类，那么就要重用该包中所有的类，这属于共同重用原则（Common Reuse Principle，CRP）。

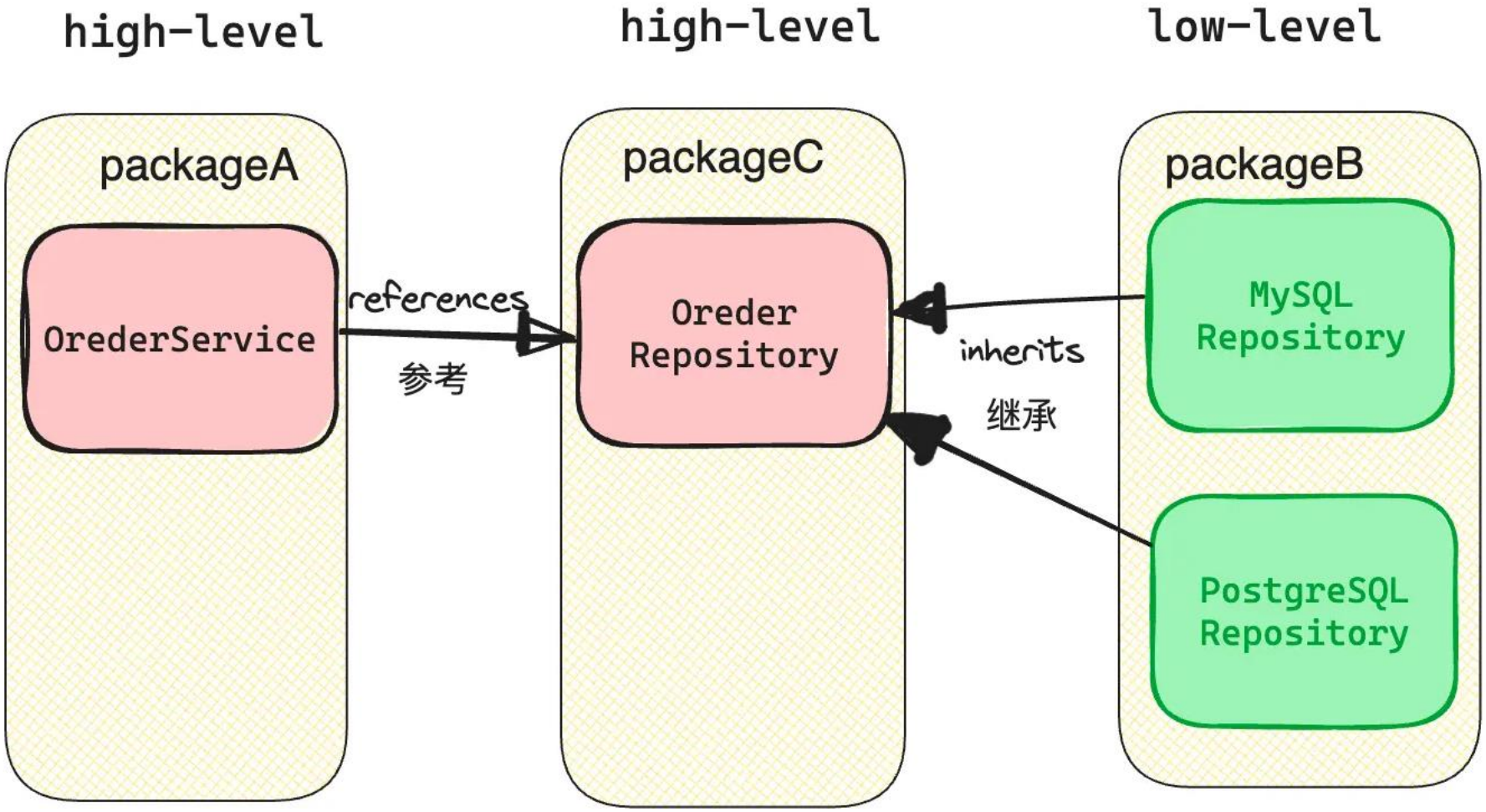
5.5 如何实现依赖倒置？- 方案二的基本思路

- 方案二：更灵活的解决方案是将抽象组件提取到一组独立的包/库中，如下图：



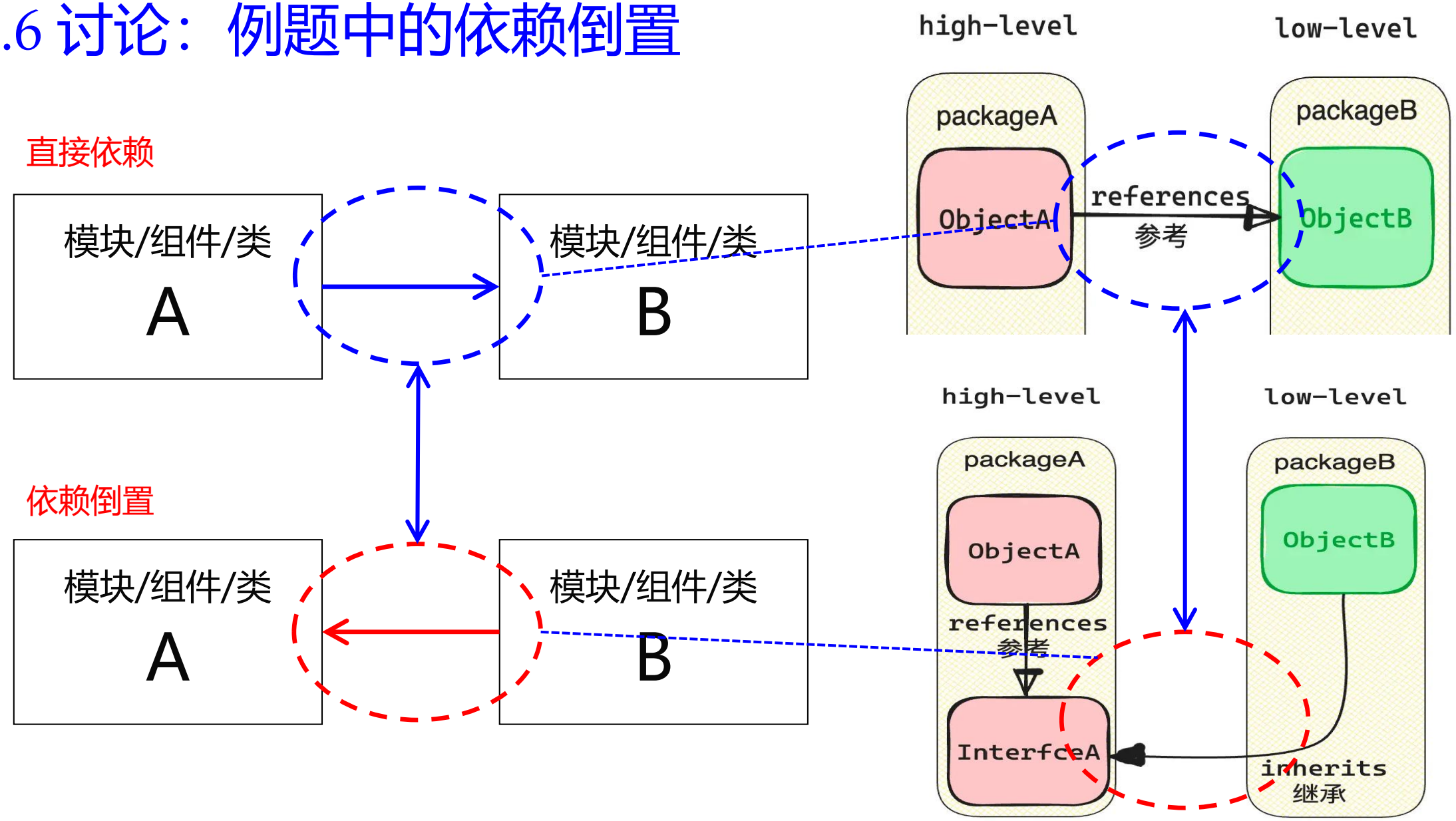
两种方案的核心思想都是一样的，只是在灵活性和组件复用的考虑上略有差异

5.5 如何实现依赖倒置？- 方案二的设计



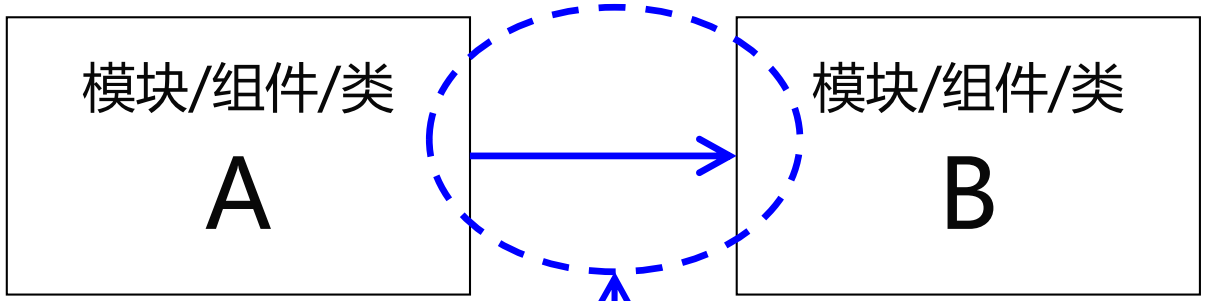
这种实现方式将每一层分离成自己的封装，鼓励任何层的再利用，提供稳健性和移动性。

5.6 讨论：例题中的依赖倒置

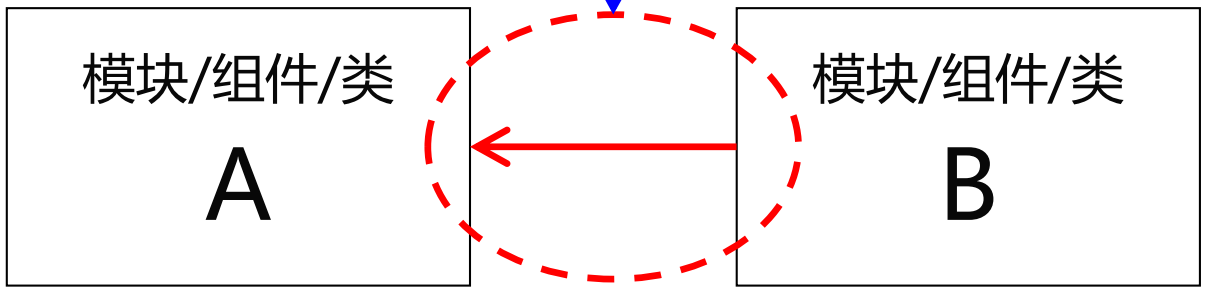


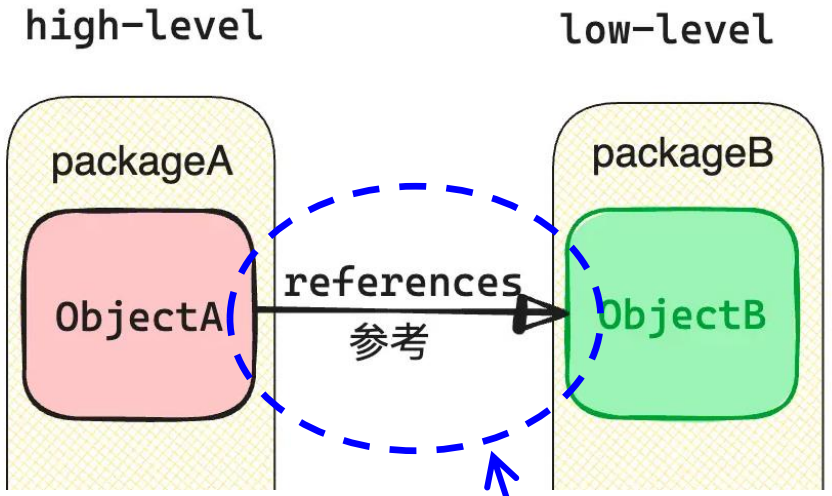
5.6 讨论：例题中的依赖倒置

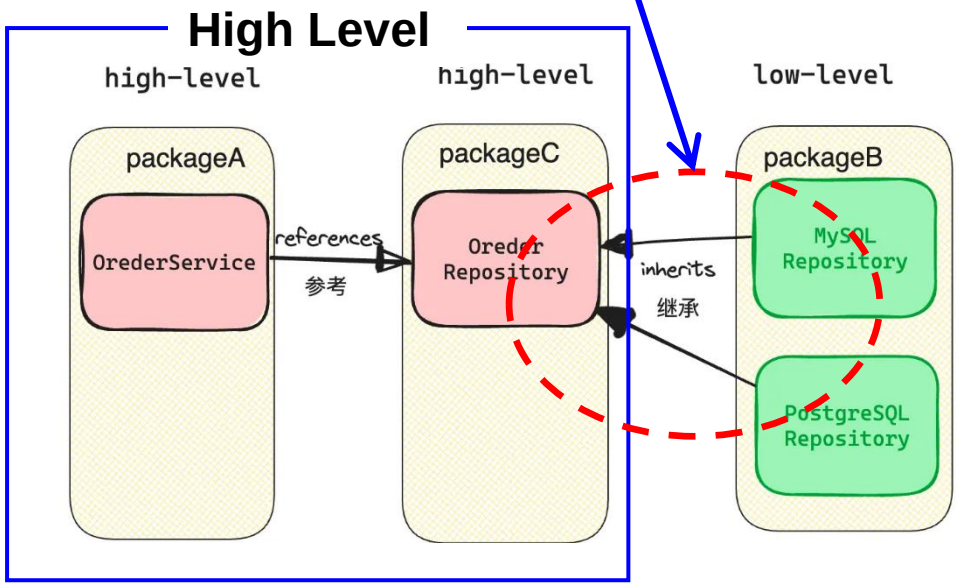
直接依赖



依赖倒置

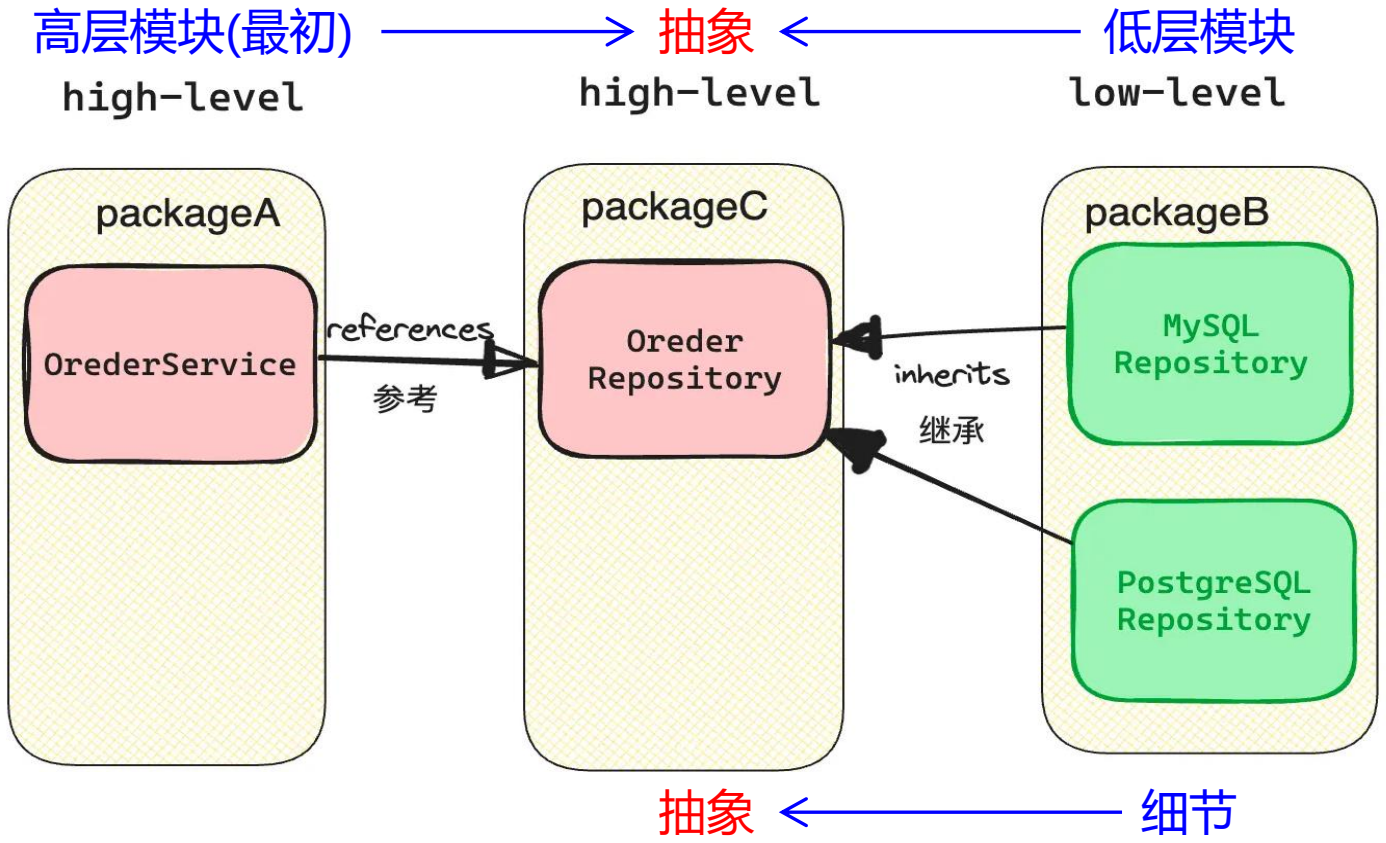






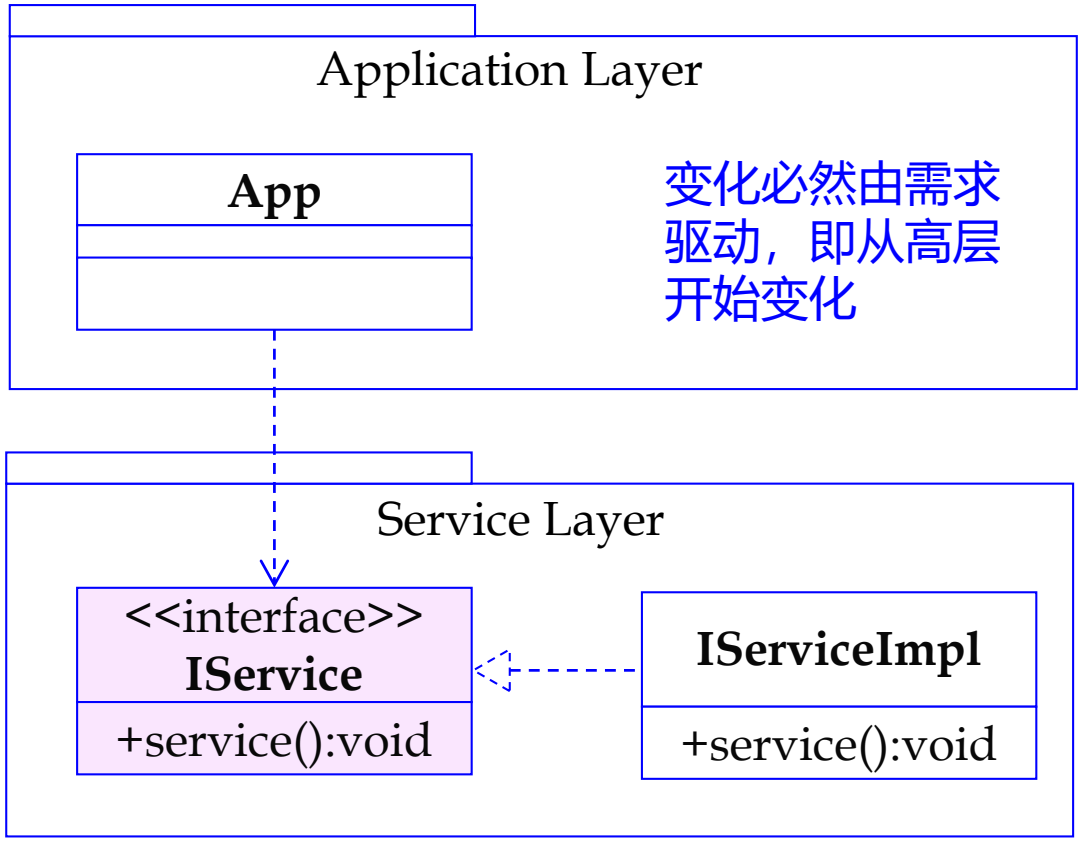
5.7 讨论：例题的设计方案与依赖倒置原则表述的对应

DIP：①高层模块不应该依赖低层模块，两者都应该依赖于抽象。②抽象不应该依赖于细节，细节应该取决于抽象



5.8 讨论：抽象接口划分到高层模块是理解依赖倒置的关键

- 按照常理，接口与其实现（类）应该处于一层或模块之中。如下图所示



当高层发生了改变，那它依赖的低层也很可能要发生变化，因为高层要完成新的业务，低层要负责提供对应的服务

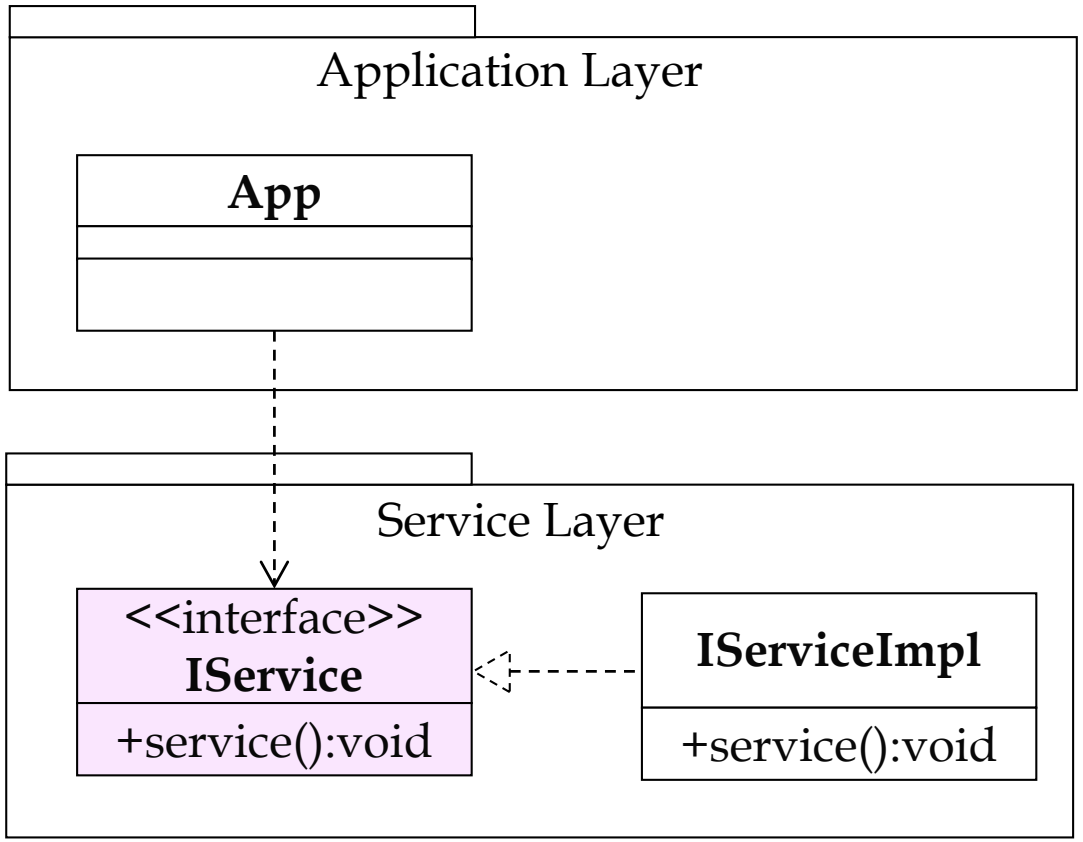
那么问题来了，谁来约定这个接口 Iservice 提供什么样的服务？

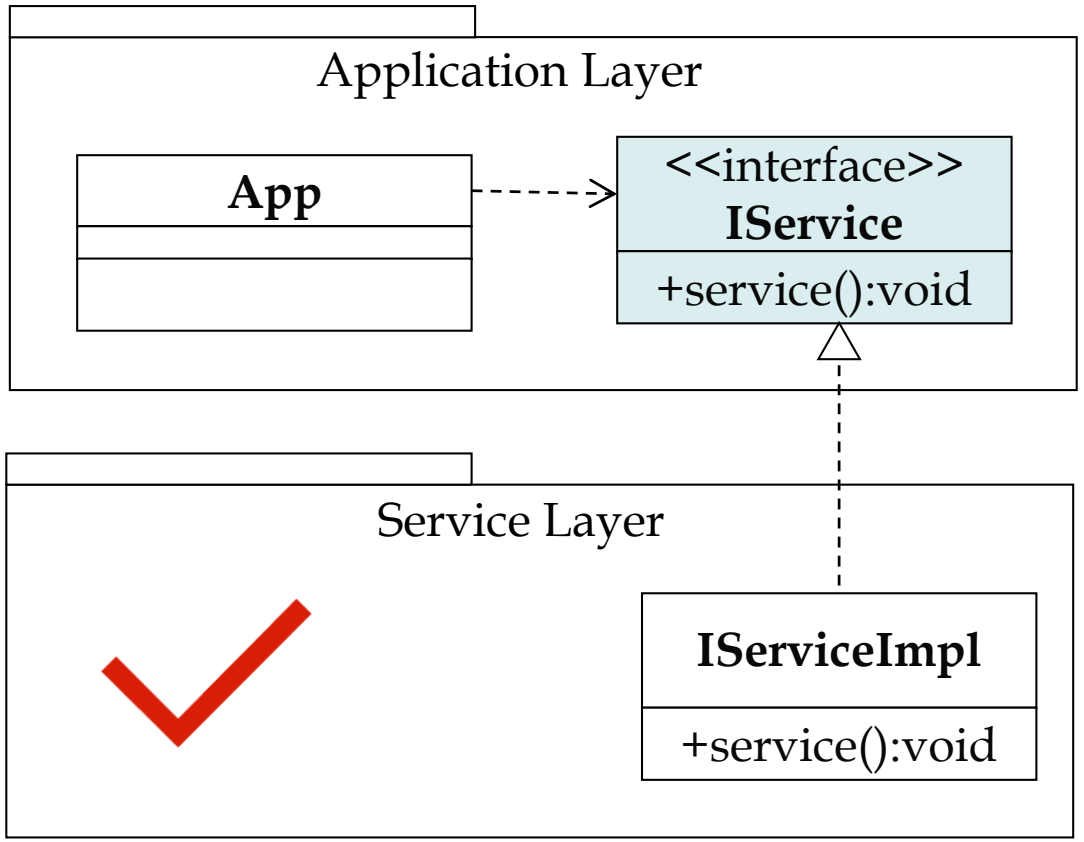
接口和实现放在低层中，那应该由低层提供，可是低层开发人员并不负责高层的应用逻辑

负责高层实现的开发人员，他们拥有需求，但无法定义描述需求的接口；负责低层的开发人员，他们不管需求，只应提供具体实现，却要维护和应用需求有关的接口。

5.8 讨论：抽象接口划分到高层模块是理解依赖倒置的关键

- 因此，将本应放在低层的接口放在高层，低层的实现依赖高层提供的接口，去实现相应的服务



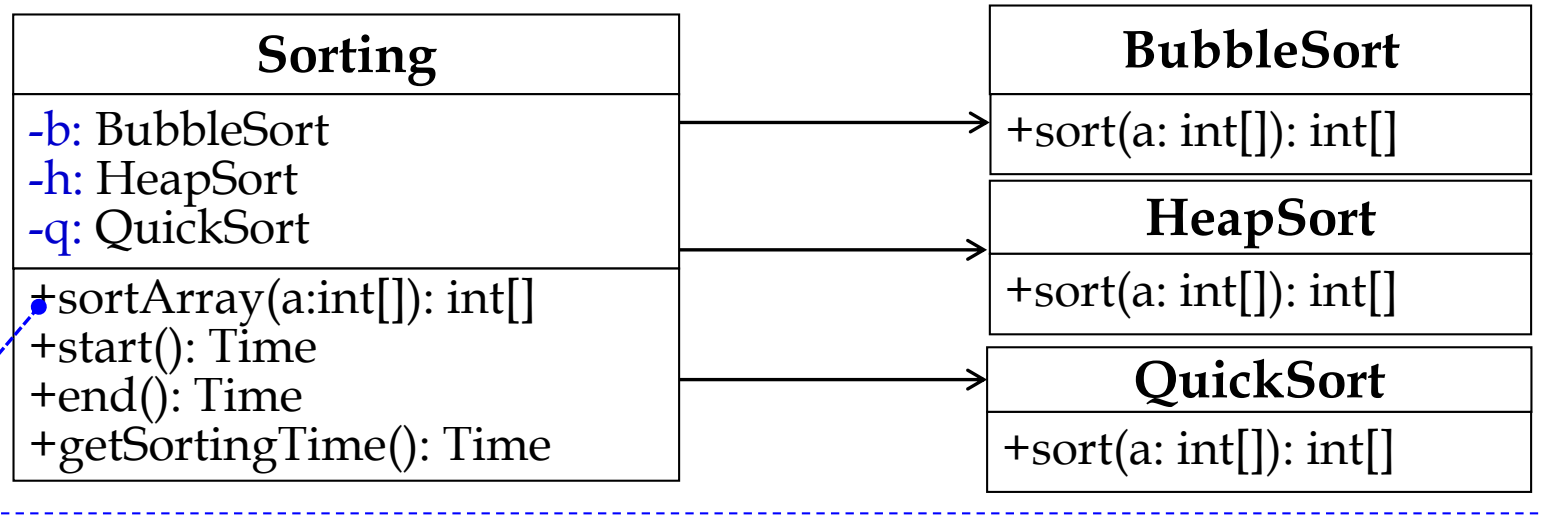


5.9 依赖倒置原则练习

- 假设需要设计了一款排序软件，对排序算法的速度进行比较。首先实现以下算法：
  - 冒泡法（bubble sort）排序
  - 堆排序法（heap sort）排序
  - 快速（quick sort）排序

5.9 依赖倒置原则练习

- A同学的设计：



sortArray 方法中含有很多条件语句，判断到底是BubbleSort，HeapSort还是 QuickSort，然后再调用相应的类的sort方法。

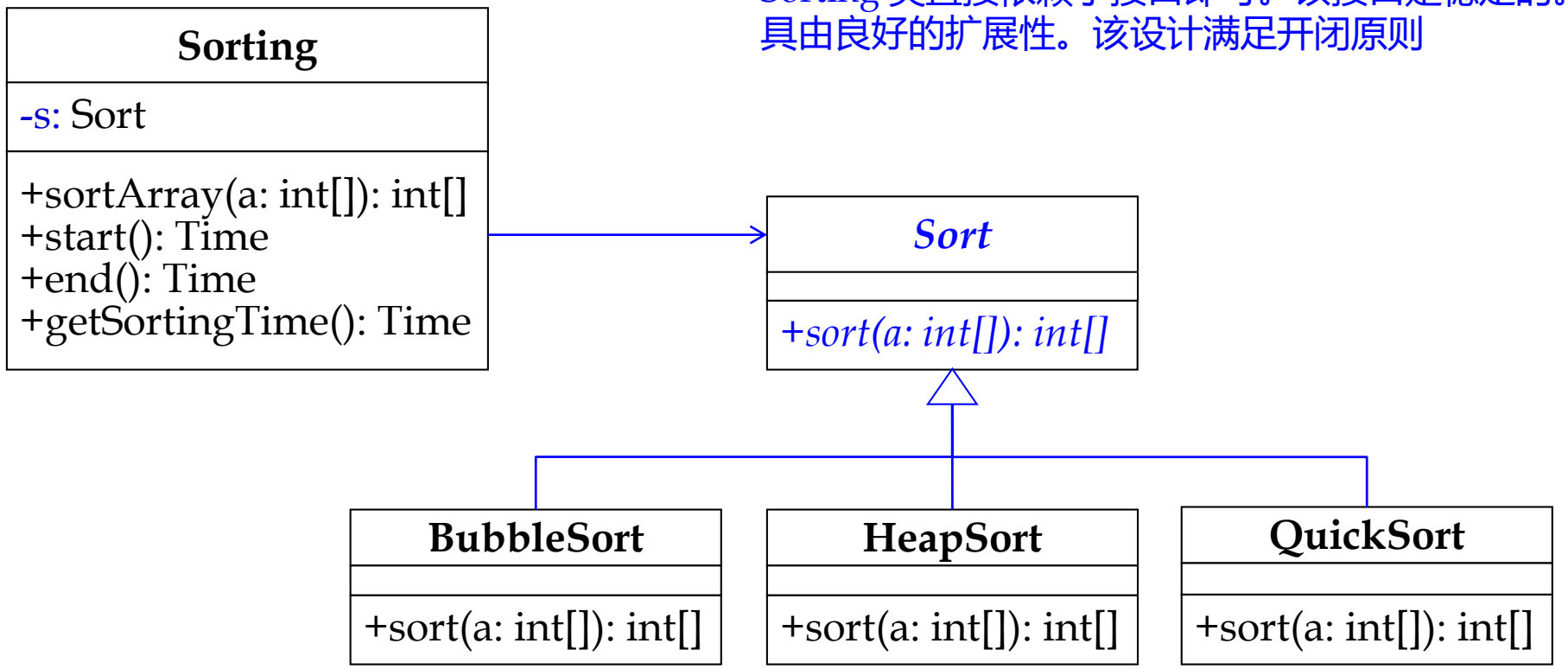
随着时间推移，会有更多的新型排序算法加入到 Sorting中，该类将频繁增加条件语句，增加新排序算法需要修改 Sorting 类，难以扩展与维护，可扩展性差。

请对 A 同学的设计进行重构，使排序软件满足开闭原则。

5.9 依赖倒置原则练习

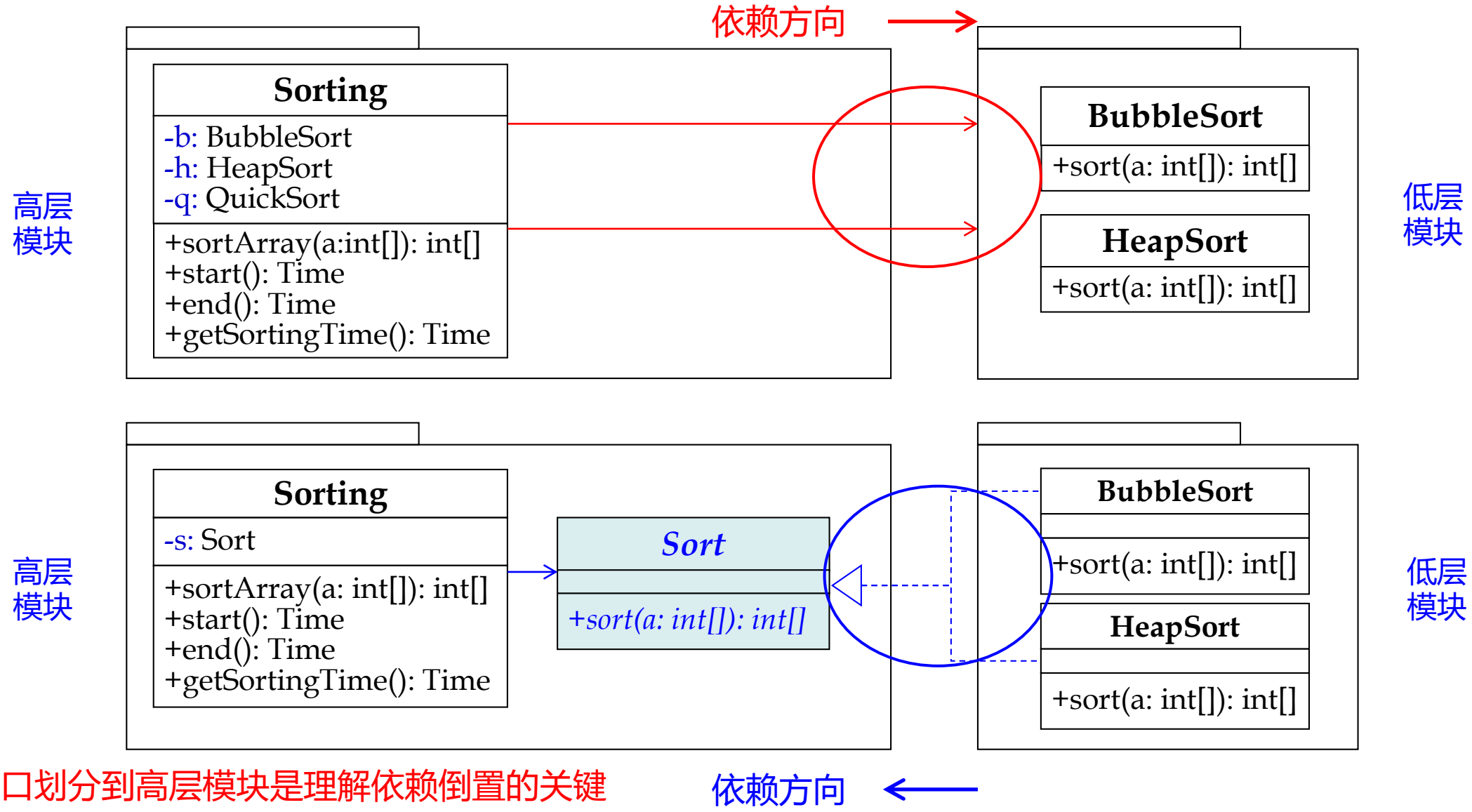
- 重构如下：

Sorting 类直接依赖于接口即可。该接口是稳定的。从而具由良好的扩展性。该设计满足开闭原则



将具体类进行抽象，形成类的层次结构。这样就可以使用多态，用到对象的时候，再指定对象类型

5.9 依赖倒置原则练习


依赖方向
高层模块
Sorting
-b: BubbleSort
-h: HeapSort
-q: QuickSort
+sortArray(a:int[]): int[]
+start(): Time
+end(): Time
+getSortingTime(): Time
BubbleSort
+sort(a: int[]): int[]
HeapSort
+sort(a: int[]): int[]
低层模块
高层模块
Sorting
-s: Sort
+sortArray(a: int[]): int[]
+start(): Time
+end(): Time
+getSortingTime(): Time
Sort
+sort(a: int[]): int[]
BubbleSort
+sort(a: int[]): int[]
HeapSort
+sort(a: int[]): int[]
低层模块
接口划分到高层模块是理解依赖倒置的关键
依赖方向

5.9 依赖倒置原则练习

- 重构后Sorting 仅仅依赖于Sort接口，符合依赖倒转原则、开闭原则
  - 可扩展性好：增加一个新算法， Sorting 类不需要修改代码；
  - 可维护性好：哪个算法需要修改代码，只需要修改相应的子类即可，对其他的类没有影响

本章总结

	设计原则	一句话归纳	目的
S	单一职责原则 Single Responsibility Principle，SRP	类的职责要单一，不能将太多的职责放在一个类中	提高类的内聚性，实现代码的高内聚、松耦合
O	开闭原则 Open Closed Principle，OCP	软件实体对扩展是开放的，但对修改是关闭的，即在不修改一个软件实体的基础上去扩展其功能	降低新风险
L	里氏代换原则 Liskov Substitution Principle，LSP	子类对象能够替换程序中父类对象出现的任何地方，并且保证原来程序的逻辑行为不变及正确性不被破坏。	防止继承泛滥
I	接口隔离原则 Interface Segregation Principle，ISP	各个类建立自己的专用接口，而不是建立万能接口	功能解耦
D	依赖倒置原则 Dependence Inversion Principle，DIP	面向接口编程，而不是面向实现编程	有利于结构的升级扩展